HorrorPlace: 100 Session and Runtime Research Questions v1

This document lists 100 focused research and implementation questions to guide AI-Chat tools, coding agents, and engine teams when wiring session logic, director personas, history-aware selectors, and runtime encounters across the HorrorPlace VM-Constellation.

Each question is tagged by primary focus:

- `[TECH]` – technical implementation and wiring
- `[DESIGN]` – design methodology, metrics, and contracts
- `[RUNTIME]` – runtime behavior, encounters, and pacing

Numbering is global (1–100) for easier cross-referencing.

1. Technical Implementation & Wiring (1–40)

1. `[TECH] (Engine)` How should `H.Director.loadPersona` cache director persona envelopes per `sessionId` so that repeated calls from AI-Chat and engine scripts remain cheap while still reflecting live consent and budget changes mid-session?
2. `[TECH] (Engine)` What minimal Lua table shape should `H.Metrics.snapshot(sessionId)` expose so that selector, director, and image-helper modules can all read UEC, EMD, STCI, CDL, and ARR consistently without each defining its own metric view?
3. `[TECH] (Engine)` How can the H. runtime surface enforce the rule "no behavior without `QueryHistoryLayer → SetBehaviorFromInvariants`" in Lua, e.g., via a central `H.requireInvariants(regionId, tileId)` helper that must be called before any mood, selector, or node logic executes?
4. `[TECH] (Engine)` What is the best pattern for sharing selector rules across AI-Chat and engine runtime: a Lua loader over `history-selector-rule-v1.json`, or an HTTP-backed schema cache, and how can we prove both surfaces stay in lockstep with Constellation-Contracts?
5. `[TECH] (Engine)` How can `H.Node.describe` be implemented so that it pulls CIC, AOS, SHCI, and LSG from the history layer without ever exposing raw region tables, and still returns enough detail for AI-Chat to narrate why a node feels liminal or heavy?

Below are focused answers for questions 1–5, aligned with the existing director-persona, metrics, selector, and node specs you already have wired into Constellation-Contracts, Horror.Place, and Codebase-of-Death.[1] [2] [3]

1. Caching H.Director.loadPersona per sessionId

H.Director.loadPersona should maintain a small, explicit in-memory cache keyed by `sessionId`, holding the fully computed "effective persona envelope" that merges persona bands, consent caps, budget caps, and comfort-policy constraints. The cache value should look structurally like the existing example result (`directorPersonaId`, `directorKind`, `toneLabel`, `invariantEnvelope`, `metricEnvelope`, `comfortPolicy`, plus resolved style profile), and must never expose any raw vault tables—only the derived, runtime envelope.[2] [1]

To keep repeated calls cheap but still reflect live consent/budget changes, `loadPersona` should split behavior into two layers: (a) a "cold" path that fetches and validates persona + comfort policy, and (b) a "warm" path that re-derives only the intersected metric/invariant bands using current consent and budget snapshots. Concretely: the cache entry holds the immutable persona contract and comfort policy, plus the last computed effective envelopes and a revision tag; when `H.Director.constrainMetrics` or any other caller detects that consent or budget revision stamps have changed (e.g., a monotonic `consentVersion` / `budgetVersion`), it asks `loadPersona` for a refresh, which recomputes the envelopes but keeps the same persona and comfort-policy references.[1] [2]

This keeps AI-Chat and engine scripts on a cheap "read effective envelope" path most of the time, while still guaranteeing that a mid-session consent downgrade or budget soft-cap will be reflected as soon as a new turn calls into director logic. Errors like `DIRECTORNOTFOUND`, `CONSENTINCOMPATIBLE`, or `COMFORTPOLICYNOTFOUND` should clear the cache entry and force callers to fall back to a non-horror or "safe narrator" persona, never leaving a stale envelope bound to the session.[2] [1]

2. Minimal Lua table from H.Metrics.snapshot(sessionId)

`H.Metrics.snapshot(sessionId)` should return a narrow, engine- and AI-Chat-friendly table that mirrors the metrics envelope semantics you already use in director personas and session-route telemetry, rather than exposing any raw time-series or derivation details. A consistent shape would be:[1] [2]

```
local snap = H.Metrics.snapshot(sessionId)

-- Expected fields
snap.ok             -- boolean
snap.error           -- nil or { code = "...", message = "..." }

snap.data = {
  -- scalar snapshot per metric, nullable when not yet defined
  metrics = {
    UEC  = 0.62,   -- or nil if not yet estimated
    EMD  = 0.41,
    STCI = 0.55,
    CDL  = 0.47,
    ARR  = 0.74,
  },

  -- optional band context, matching the currently effective bands
```

```
    bands = {
      UEC  = { min = 0.4, max = 0.7 },
      EMD  = { min = 0.1, max = 0.3 },
      STCI = { min = 0.4, max = 0.7 },
      CDL  = { min = 0.5, max = 0.7 },
      ARR  = { min = 0.5, max = 0.7 },
    },

    -- optional provenance
    source = {
      directorPersonaId = "dir-cold-overseer-01",
      aiChatProfileId   = "acp-...",
      runContractId     = "run-...",
    }
  }
```

This matches the pattern already used in `H.Director.constrainMetrics`, where effective metric bands and clamped values are returned together and treated as authoritative by persona and selector code. Selector modules, node describers, and image-helpers then only need to depend on `snapshot.data.metrics` for current values and optionally on `snapshot.data.bands` when they want to make relative decisions (e.g., push UEC up toward band center), instead of rolling their own metric views.[2] [1]

When no metrics are yet available for the session, `H.Metrics.snapshot` should still return `ok = true` with `metrics` fields set to `nil` and `bands` populated from the current persona/run envelopes, so callers can explicitly distinguish "no data yet" from "metric is low (0.0)". This keeps `H.Node.describe` and history-selector telemetry consistent with the existing "nil means unknown, not zero" doctrine.[4] [2]

3. Enforcing "no behavior without QueryHistoryLayer → SetBehaviorFromInvariants"

The clean way to enforce this rule in Lua is to centralize invariant access behind a small `H.Invariants` surface and require all behavior-relevant modules to call a helper like `H.requireInvariants(regionId, tileId, context)` before they perform any selection, mood, or node logic. The enforcement has two layers: static and runtime.[1] [2]

At static/CI time, Codebase-of-Death should ship a linter that rejects any Lua file which accesses history or invariant tables directly (e.g., `history[...]`, `region.invariants`) rather than calling `H.requireInvariants` or the higher-level `H.Invariants.sample` helper. This matches the existing pattern where any direct access to vault tables or unapproved modules fails CI.[2]

At runtime, `H.requireInvariants(regionId, tileId, ctx)` should:

1. Look up or compute the invariant snapshot for the requested `(regionId, tileId)` via `H.Invariants.sample`, including CIC, MDI, AOS, RRM, FCF, SPR, RWF, DET, HVF, LSG, SHCI.[5] [1]

2. Attach this snapshot to the provided `ctx` (or a new context) and tag it with a monotonic `invariantsVersion` stamp.[1]
3. Record a structured telemetry event "invariants-queried" so later analysis can verify that any selector or director decision was preceded by an invariant fetch.[2]

Any selector, director, node, or banter function that requires invariants should either accept this enriched context or call `H.requireInvariants` at the top and short-circuit with a structured error if invariants cannot be loaded (e.g., `HISTORYLAYERUNAVAILABLE`). Over time you can make this stricter by having helper wrappers such as `H.Selector.choosePatternSafe` and `H.Node.choosenextSafe` that always perform the `requireInvariants` step internally, and deprecate the unsafe variants in CI, so engine and AI-Chat code cannot accidentally skip the history layer.[4] [1] [2]

4. Sharing selector rules across AI-Chat and engine runtime

Given the existing contract-first design for `history-selector-pattern` and `history-selector-rule` schemas, the most robust pattern is:

- Treat `history-selector-rule-v1.json` and `history-selector-pattern-v1.json` as the single source of truth in HorrorPlace-Constellation-Contracts, and build both AI-Chat and engine selector behavior strictly on top of these schemas.[2]
- At runtime, have a small selector-rule loader in each environment: a Lua loader in Horror.Place / Codebase-of-Death that reads pre-compiled rule/pattern JSON (or generated Lua tables), and a Rust/Go/Node HTTP service (or static file cache) that AI-Chat orchestrators query to get the same rule/pattern catalog.[4] [2]

For sharing rules, a file-based schema cache is preferable inside the repos themselves (Lua loader over versioned JSON or Lua-generated tables), while AI-Chat can talk to an HTTP mirror that simply exposes the same validated catalog via IDs and schema-checked JSON. Lockstep is enforced at the governance layer: CHATDIRECTOR and CI must declare a canonical rules "catalog version" (e.g., via a hash or version field in a `selector-rule-catalog` manifest), and both the HTTP surface and the Lua side must refuse to operate if their local catalog hash does not match the manifest for the current phase/tier.[6] [2]

Concretely, you can:

- Add a `selectorRuleCatalogId` and `catalogHash` field to the runtime selector config (`history-selector-envelope`), and require both AI-Chat and engine to log this pair in every `history-selector-decision` telemetry event.[4] [2]
- Add a CI job that validates that the exported Lua selector tables in Codebase-of-Death are generated from exactly the same `history-selector-rule` and `history-selector-pattern` JSON files that back the AI-Chat HTTP catalog (matching `catalogHash`).[6] [2]

This gives you a provable, machine-checkable link: one rules catalog in Constellation-Contracts, a single catalog hash, and both AI-Chat and engine loaders agreeing on that hash before any selection runs.[2]

5. Implementing H.Node.describe without exposing raw region tables

`H.Node.describe` should act as the canonical "node view" for AI-Chat and engine tools, assembling a synthesized description from contracts and invariant/metric snapshots, but never leaking raw history tables or vault-internal structures. Implementation-wise, it should look like:[4] [1]

1. Load the node contract via the narrow `H.Contract.load("dungeonNodeContract", nodeId)` helper, which already enforces region consistency and hides registry details.[4] [2]
2. Fetch local invariants via `H.Invariants.sample(regionId, tileId)` to get CIC, AOS, SHCI, LSG and related fields.[5] [1]
3. Optionally fetch `H.Metrics.snapshot(sessionId)` to attach current UEC/EMD/STCI/CDL/ARR if available, following the nil-means-unknown rule.[1] [4]
4. Compute derived, narrative-oriented fields like `liminalityScore`, `weight`, or `feelsLike` from CIC, AOS, SHCI, and LSG using a documented mapping, instead of surfacing the raw invariant table.[4] [1]

The returned table should be a sanitized, documented shape such as:

```
local result = H.Node.describe(regionId, nodeId, { sessionId = sessionId })

-- result.data:
{
  nodeId      = "node-stairwell-02",
  regionId    = "reg-...",
  role        = "liminalthreshold",   -- from node contract
  liminalTags = { "liminalstairwell" },

  invariants = {
    CIC  = 0.4,
    AOS  = 0.3,
    SHCI = 0.84,
    LSG  = 0.7,
  },

  feels = {
    liminalIntensity = 0.7,
    historyWeight    = 0.8,
    spectralTension  = 0.75,
  },

  metrics = {
    UEC  = 0.62,
    EMD  = 0.41,
    STCI = 0.55,
    CDL  = 0.47,
    ARR  = 0.74,
  },

  topology = {
    connections = {
      { targetNodeId = "node-maintenance-03", hidden = false, oneWay = false },
```

```
      { targetNodeId = "node-shaft-01",       hidden = true,  oneWay = true  },
    }
  },

  hints = {
    explanation = "Liminal stairwell with strong spectral coupling and moderate drea
    reasons     = { "liminalTags", "SHCI", "LSG" },
  }
}
```

Notice that `invariants` is a small, explicit subset, not a raw region table, and everything else (like `feels` and `hints`) is derived. Security-wise, when called from AI-Chat, `H.Node.describe` must also enforce session-scoped visibility rules: confirm the node belongs to the current `dungeonRunContract` when `sessionLock = true`, redact sensitive fields based on tier/consent, and return `FORBIDDEN` or `NODENOTFOUND` if a caller attempts to enumerate nodes outside its allowed graph.[1] [4]

This gives AI-Chat enough information to narrate *why* a node feels liminal or heavy—"high SHCI and LSG, liminal stairwell tag"—while keeping the underlying region/tile history layer encapsulated behind `H.Invariants` and registry loaders.[3] [4] [1]

⁂

6. `[TECH] (Engine)` What standard Lua interface should dungeon node contracts implement (e.g., `NodeContract.get_connections(nodeId)`) so that `H.Node.score_next_from_history` can operate identically in Death-Engine, Godot, and Unreal bindings?

7. `[TECH] (Engine)` How should `H.Selector.selectPattern` be extended to accept an optional `directorPersonaId` and `experienceContractId` while preserving backward compatibility with existing selectors that do not know about director personas yet?
8. `[TECH] (Safety)` Which shared error-code schema should all H. modules use (`SESSION_NOT_FOUND`, `CONSENT_INCOMPATIBLE`, `BUDGET_EXCEEDED`, etc.), and how will CI enforce that Lua and Rust implementations return only codes declared in that schema?
9. `[TECH] (Engine)` What is the minimal metadata contract for exposing region-level invariants to shaders (e.g., for screen-space FX) via a `H.Invariants.toRenderParams(regionId, tileId)` helper without leaking any internal registry details?
10. `[TECH] (Engine)` How can we design a simple, engine-agnostic `H.Audio` Lua façade that maps CIC, AOS, DET, and LSG into a small set of RTPCs (e.g., `horror_pressure`, `liminal_intensity`) which Death-Engine and external engines can all implement in their own audio middlewares?

Answers below stay aligned with the existing dungeon-node, selector, invariants, and audio research you already have in Constellation-Contracts and Codebase-of-Death.[22] [23] [24] [25]

6. Standard Lua interface for dungeon node contracts

Dungeon node contracts should present a single, engine-neutral accessor surface that hides registries and raw tables and that H.Node can rely on identically in Death-Engine, Godot, and Unreal. You already sketched this as a narrow loader plus a describe helper; the key is to standardize the "node contract view" used by scoring:[23]

- A contract loader: `H.Contract.load("dungeonNodeContract", nodeId)` returning the node card (role, `topology.connections`, `liminalTags`, `mechanicHooks`, `invariantBands`, `metricTargets`).[24] [23]
- A topology accessor: `NodeContract.get_connections(nodeId)` as a thin wrapper that calls the loader and returns `topology.connections` as a simple array of `{ targetNodeId, kind, hidden, oneWay }`, already normalized so scoring code never touches the raw contract.[22] [23]
- A "node view" accessor used by `H.Node.scorenextfromhistory`: `NodeContract.describe(nodeId)` returning a stable, read-only table with `role`, `liminalTags`, `mechanicHooks`, and flattened invariant bands (`CIC`, `AOS`, `DET`, `LSG`, `SHCI`, `HVFmag`) ready for scoring.[23]

In each engine, the bindings (Godot C++, Unreal C++) only need to bridge into this Lua surface: they never parse NDJSON or JSON themselves and never read registries directly. This keeps `H.Node.scorenextfromhistory` identical across engines: it always works against `NodeContract.describe` and `NodeContract.get_connections`, with the underlying loader adapted per engine but hidden behind the same Lua signatures.[22] [23]

7. Extending H.Selector.selectPattern with persona/experience IDs

To add `directorPersonaId` and `experienceContractId` while staying backward-compatible, `H.Selector.selectPattern` should accept an extended `context` table where these fields are optional, and treat their absence as "use legacy behavior."[23] [22]

Today the signature is effectively:

```
H.Selector.selectPattern(sessionId, regionId, context)
```

You can evolve `context` to include:

```
context = {
  descriptorId        = "desc-...",
  consentTier         = "adult-basic",
  routerStateId       = "rtr-...",
  budgetSnapshot      = { ... },
  banterPlan          = { ... },

  -- New, optional
  directorPersonaId   = "dir-cold-overseer-01",  -- optional
```

```
  experienceContractId = "exp-guided-ritual-01", -- optional
}
```

Selector rules and patterns then may:

- Narrow candidate patterns based on persona and experience envelopes when these IDs are present (e.g., only consider patterns whose entertainment targets lie within the persona's `metricTargets` and the experience contract's bands).[24] [23]
- Fall back to existing invariant- and budget-only selection when they are absent (e.g., for older engines or tools that never populate these fields).[24]

On the telemetry side, `history-selector-decision` events should add optional `directorPersonaId` and `experienceContractId` fields so AI-Chat and offline analysis can correlate particular personas/experiences with selector outcomes, without making them required for older producers. CI can enforce that selector code must treat these IDs as hints—never as mandatory keys—by linting for presence checks that are always guarded with nil-safe logic.[22] [24]

8. Shared error-code schema for all H. modules

All H. modules should use a single, centralized error-code registry schema in HorrorPlace-Constellation-Contracts (e.g., `schemas/runtime/h-error-codes.v1.json`) that defines canonical codes like `SESSIONNOTFOUND`, `CONSENTINCOMPATIBLE`, `BUDGETEXCEEDED`, `PATTERNNOTFOUND`, etc. Each error entry would specify:[25] [24]

- `code`: canonical string (no repo-local variants).
- `module`: primary owner (e.g., `H.Consent`, `H.Budget`).
- `severity` and `category`: for tooling and telemetry.
- Optional `fallbackAction`: hints like `downshift`, `refuse`, `retry`.[24]

Lua and Rust APIs must adopt a standard error envelope: `{ code = "SESSIONNOTFOUND", message = "...", details = { ... } }`, and schema docs for each module should constrain their `error.code` fields to the set enumerated in `h-error-codes.v1.json`.[25] [24]

CI enforcement then has two layers:

1. **Static schema checks**: all public API schemas (Lua envelopes, telemetry events, JSON RPC responses) use `enum` / `$ref` into `h-error-codes.v1.json` for `error.code` fields, so invalid codes fail schema validation at build time.[24]
2. **Linter checks**: Codebase-of-Death and Spectral-Foundry include linters that scan Lua and Rust for string literals assigned to `error.code` or returned in `error` envelopes; any literal not present in the error-code registry is rejected.[25] [24]

This guarantees that every error observable by AI-Chat, engines, or telemetry is taken from a single, versioned schema, and prevents silent proliferation of ad-hoc codes like `BADSESSION` or `UNKNOWN1` in individual modules.[24]

9. Minimal metadata contract for H.Invariants.toRenderParams

`H.Invariants.toRenderParams(regionId, tileId)` should expose a minimal, shader-ready parameter block that derives from CIC, AOS, DET, HVF, LSG, etc., but never reveals IDs, raw history, or registry shapes. A reasonable contract is:[23] [22]

```
local params = H.Invariants.toRenderParams(regionId, tileId)

-- Example output:
params = {
  horror_pressure    = 0.0,   -- 0-1, derived mainly from DET and CIC
  spectral_density   = 0.0,   -- 0-1, from SHCI and SPR
  liminal_intensity  = 0.0,   -- 0-1, from LSG and HVFmag
  archival_fog       = 0.0,   -- 0-1, from AOS and AOS-linked AOS/MDI bands
  history_noise      = 0.0,   -- 0-1, from AOS + FCF where archives contradict
}
```

The mapping from invariants to these normalized floats should live as a small policy config in Constellation-Contracts (e.g., `schemas/runtime/invariants-render-policy.v1.json`), not hardcoded in Lua, so you can tune how CIC vs AOS influences `horror_pressure` without engine changes. Engines then bind these values to their local RTPC/uniform names (`u_horror_pressure`, `rtpc_spectral_density`, etc.) but never see region IDs, event lists, or contract details.[25] [22] [23]

By treating `toRenderParams` as the *only* approved way to derive horror-related shader parameters from the invariant layer, you keep visual effects grounded in the same numeric spine while preserving strict encapsulation of history data.[23] [25]

10. H.Audio Lua façade mapping invariants to RTPCs

The `H.Audio` façade should provide a small, engine-agnostic API that converts invariants (and optionally metrics) into a few normalized RTPC-style channels, leaving each engine free to map those channels into its own audio middleware. A minimal design:[22] [23]

- Function: `H.Audio.computeParams(regionId, tileId, sessionId?)`
  Returns a table with fields like:

  ```
  {
    horror_pressure    = 0.0,  -- from DET, CIC, MDI
    liminal_intensity  = 0.0,  -- from LSG, HVFmag
    spectral_activity  = 0.0,  -- from SHCI, SPR
    comfort_relief     = 0.0,  -- inverse of DET and CDL bands (higher in safe hubs
  }
  ```

- Optional function: `H.Audio.apply(engineAdapter, params)` that calls a thin engine bridge, e.g., `engineAdapter.setRTPC("horror_pressure", params.horror_pressure)`, keeping all engine specifics out of the core Lua.[25] [22]

Internally, `H.Audio.computeParams` should call `H.Invariants.toRenderParams` and possibly `H.Metrics.snapshot(sessionId)` to incorporate both slow-changing history (CIC, AOS, SHCI,

LSG) and session state (current DET or CDL trajectory). The exact mapping from invariants/metrics to audio channels again lives in a small policy schema (e.g., `schemas/runtime/audio-invariant-mapping.v1.json`), ensuring it is tunable and CI-checked for valid ranges.[22] [23] [25]

Death-Engine, Godot, and Unreal integrations then:

- Bind the same four logical RTPCs (`horror_pressure`, `liminal_intensity`, `spectral_activity`, `comfort_relief`) in their audio graphs.
- Call into `H.Audio.computeParams` once per logical tick or "audio update" and push the resulting floats into Wwise/FMOD/custom parameters.[22]

Because `H.Audio` never exposes IDs or history tables and only outputs normalized floats, it is safe to use in Tier-1 repos while still guaranteeing that every swell of sound is driven by the same invariant and metric spine as selectors and nodes.[23] [25] [22]

⁂

11. `[TECH] (CI)` What static analysis rules should be added to Codebase-of-Death so that any Lua script accessing invariants or contracts outside the H. API (e.g., reading raw JSON tables) fails CI by default?

12. `[TECH] (CI)` How should Constellation-Contracts define a reusable CI job that validates that every new history-selector rule, pattern, or director persona references only invariants and metrics declared in the canonical spine, never ad-hoc fields?
13. `[TECH] (Authoring)` What JSON structure should AI-Chat use as a "session planning skeleton" that lists planned objectKinds, targetRepos, tiers, and invariants/metrics touched, and how can CHATDIRECTOR verify that all subsequent authoring responses stay within that plan?
14. `[TECH] (Authoring)` How can we expose a lightweight CLI tool that takes a single session plan JSON, validates it against `ai-safe-authoring-contract`, and prints a concise summary for human designers before any AI or code runs?
15. `[TECH] (Engine)` What is the cleanest way for Godot and Unreal plugins to present H. functions: directly exposing Lua functions through bindings, or wrapping them in native functions that internally call into an embedded Lua VM?

11–15 all build directly on the existing spine and research objects; below are concrete answers and wiring strategies.[26] [27] [28] [29]

11. Static analysis rules in Codebase-of-Death

Static analysis should enforce a simple doctrine: Lua code may only touch invariants, metrics, and contracts through the H. API and approved adapter modules, never via raw JSON/NDJSON tables or repo-local loaders.[27] [26]

In practice you want:

- **Approved import whitelist:** allow `require("hpcruntime")`, `require("horrorinvariants")`, `require("horrorcontracts")`, etc., but flag any `require` of low-level JSON readers or registry blobs (`require("black_archivum")`, `require("raw_region_tables")`, `require("cjson")` in engine code) in Codebase-of-Death. The linter rule is "no direct JSON/NDJSON decode from repo data; only via H.* loaders."[28]
- **Banned field access patterns:** scan Lua for property accesses like `.CIC`, `.AOS`, `.SHCI`, `.UEC`, `.ARR`, `.invariantBands`, `.metricTargets` on variables that did not originate from H. APIs (e.g., not from `H.Invariants.*`, `H.Metrics.*`, `H.Node.describe`, `H.Dungeon.sampletile`). If a script parses JSON itself and then reads these fields, CI fails.[29] [26]
- **Contract access guard:** enforce that the only allowed contract loaders in engine/runtime Lua are the narrow helpers (`H.Contract.load`, `Contracts.node`, etc.). Any function named `load_*_contract`, `read_contract_json`, or direct `io.open` / `love.filesystem.read` paths into contracts directories should be rejected.[26] [27]
- **Module-scope annotations:** encourage modules that are allowed to see raw contracts (e.g., a single registry compiler) to carry an explicit comment/annotation recognized by the linter (`---@allow_raw_contracts`). CI then fails any raw access in files that are not whitelisted.[28]

Technically this can be implemented as a custom Lua AST linter (e.g., `luacheck` plugin or a Rust tool in the Codebase-of-Death CI pipeline) that:

1. Builds a symbol table of variables populated via H.* calls.
2. Flags invariant/metric field access on any other variable.
3. Flags forbidden `require` targets and file I/O into `schemas*/`, `contracts*/`, `registry-*` paths.[27] [28]

The result is that all engine-visible logic must call `H.Invariants`, `H.Metrics`, `H.Node`, `H.Dungeon`, etc., and cannot accidentally bypass the invariant/contract layer.[26] [27]

12. Reusable CI job for invariant/metric references

Constellation-Contracts should provide a reusable workflow (e.g., `.github/workflows/invariants-metrics-refcheck.reusable.yml`) that validates any new history-selector rule, pattern, or director persona only references invariants and metrics declared in the canonical spine schemas (`invariants.v1`, `entertainment-metrics.v1`).[27] [28]

The pattern:

- **Canonical lists:** treat `schemas/invariants.v1.json` (CIC, MDI, AOS, RRM, FCF, SPR, RWF, DET, HVF, LSG, SHCI) and `schemas/telemetry/entertainment-metrics.v1.json` (UEC, EMD, STCI, CDL, ARR) as the sole sources of valid keys.[27]
- **Ref-check script:** write a small checker (Rust or Python) that loads these canonical schemas, builds sets of allowed invariant and metric identifiers, and then inspects candidate files:
  - `schemas/selector/history-selector-rule-v1.json` and any rule instances.
  - `schemas/selector/history-selector-pattern-*.json`.
  - `schemas/runtime/director-persona-contract.v1.json` instances.[29] [27]
    The checker traverses JSON, finds all objects that look like "bands" or "weights" (e.g., keys under `invariantBands`, `historyCoupling`, `metricTargets`, `entertainmentTargets`) and verifies that each key is in the canonical sets.[29] [27]
- **Job wiring:** the reusable job should accept a `paths` glob (e.g., `schemas/selector/**`, `schemas/runtime/director-persona-*.json`) and run the checker on diffed files or on the whole tree. Any ad-hoc field (`CREEP`, `MYSTERY`, `INTENSITY2`, etc.) causes CI failure.[27]

Because the canonical schemas live in the same repo, the job is self-contained and can be reused across other repos if they mirror those schemas as read-only references.[28]

13. Session planning skeleton JSON and enforcement

For authoring, AI-Chat should start every coding/content session by emitting a **session planning skeleton** that encodes what it intends to touch; earlier research calls this an AI authoring session contract. A minimal shape, validated by `ai-safe-authoring-contract.v1.json`, could look like:[26] [27]

```
{
  "sessionId": "sess-...",
  "agentProfileId": "agent-...",
  "aiChatProfileId": "aichat-profile-...",
  "mode": "MapDesignWizard",

  "artifacts": [
    {
      "objectKind": "dungeonNodeContract",
      "targetRepo": "HorrorPlace-Constellation-Contracts",
      "path": "schemas/runtime/dungeon-node-contract.v1.json",
      "tier": "Tier1Public",
      "schemaRef": "dungeon-node-contract.v1",
      "invariants": ["CIC", "AOS", "DET", "LSG", "SHCI"],
      "metrics": ["UEC", "EMD", "STCI", "CDL", "ARR"]
    },
    {
      "objectKind": "historySelectorPattern",
      "targetRepo": "HorrorPlace-Constellation-Contracts",
      "path": "schemas/selector/history-selector-pattern-*.json",
      "tier": "Tier1Public",
      "schemaRef": "history-selector-pattern.v1",
      "invariants": ["CIC", "AOS", "SPR", "SHCI"],
```

```
      "metrics": ["UEC", "ARR"]
    }
  ]
}
```

This skeleton says: "during this session the agent may author or edit these objectKinds, in these repos, at these tiers, using only these invariant/metric axes." [26] [27]

CHATDIRECTOR can verify adherence in two ways:

1. **Plan validation upfront:** validate the skeleton against `ai-safe-authoring-contract.v1.json` and additional governance rules (e.g., agent profiles cannot write to Tier 2 repos; certain objectKinds require DeadLedger capabilities). Only if it passes does the session proceed.[28] [27]
2. **Per-turn envelope checking:** each authoring response is parsed into "proposed artifacts" (e.g., JSON blocks tagged with `objectKind`, `schemaRef`, `targetRepo`) and a validator ensures:
   - Every proposal's `objectKind`, `targetRepo`, `tier`, and `schemaRef` appear in `artifacts` of the plan.
   - Newly referenced invariants or metrics are subsets of the `invariants` / `metrics` arrays for that artifact entry.
   - No additional repos or tiers are touched.[26] [27]

If a response goes outside the plan, CHATDIRECTOR either rejects it (and asks the agent to revise) or requires an explicit plan revision that itself passes `ai-safe-authoring-contract` validation before proceeding.[27]

14. Lightweight CLI for session plans

To make this visible to humans, expose a small CLI (e.g., `hp-authoring-plan`) in HorrorPlace-Constellation-Contracts or Orchestrator that takes a plan JSON, validates it, and prints a summary.[28] [27]

The tool flow:

1. `hp-authoring-plan validate path/to/plan.json`
   - Loads `ai-safe-authoring-contract.v1.json`.
   - Validates the plan structure and values (objectKinds known, repos from the 13-repo list, tiers valid, invariants and metrics from canonical spines).[28]
2. `hp-authoring-plan summarize path/to/plan.json`
   - Prints a concise, human-readable summary, e.g.:
   - Session: id, mode, aiChatProfileId, agentProfileId.
   - For each artifact entry: objectKind, targetRepo, tier, schemaRef, count of invariants/metrics, and a short note like "nodes only, Tier1, CIC/AOS/DET/LSG/SHCI." [26] [27]

Internally this can be a Rust or Go binary distributed via Cargo or a simple Python script wired into a `make` target; the key is that CI and developers use the same tool. The same binary can be called by CHATDIRECTOR as a library or subprocess to pre-validate session plans before they are accepted.[27] [28]

15. Presenting H. functions in Godot and Unreal

For engine integration, the cleanest pattern is to **wrap H. Lua functions in thin native bridges** rather than exposing the raw Lua environment everywhere.[26] [27]

- Use an embedded Lua VM (or your existing H. runtime) as the single execution environment.
- For Godot: write a GDExtension or C++ module that registers high-level methods (e.g., `HRun.init_session`, `HRun.step`, `HNode.describe`) as Godot-callable functions, but internally each calls into the Lua VM using the canonical H. APIs defined in Horror.Place.[27]
- For Unreal: create a C++ subsystem or Blueprint function library that offers a similar surface; these functions again call into the embedded Lua H. runtime.[28] [27]

This approach has several advantages over directly exposing arbitrary Lua:

- Engine code only sees a **small, stable API** that matches the docs (`horror-engine-lua-api-v1.md`, `history-selector-lua-api-v1.md`, `director-persona-safety-adapter-v1.md`, etc.), so your H. semantics stay consistent even if Lua implementation details change.[26] [27]
- You can enforce invariants at the bridge boundary (e.g., ensuring session IDs and region IDs are present, or that only approved H. namespaces are callable from engine code).
- Static analysis in Codebase-of-Death only needs to police Lua files and the bridge implementation, not arbitrary engine projects, because engine code never manipulates invariants or contracts directly.[28] [27]

So: treat Lua as the spine implementation language and present **native facades** in Godot/Unreal that call into it, rather than embedding Lua into every gameplay script.[27] [26]

⁂

16. `[TECH] (Telemetry)` How should session metrics envelopes annotate which director persona and comfort policy were active so offline analysis can correlate different downshift strategies with UEC, CDL, and ARR trajectories?

17. `[TECH] (Engine)` What base interface should an “image-helper” module implement (e.g., `H.ImageHelper.describe(context, invariants, metrics, inventory)`) so AI-Chat, Godot, and Unreal all see the same contract for generating clue-rich prompts while keeping raw image generation out of Tier 1 repos?

18. `[TECH] (Engine)` How can H. modules expose a uniform `context` table shape (sessionId, regionId, tileId, nodeId, patternId, directorPersonaId) that all decision functions accept, so new modules remain pluggable without combinatorial parameter growth?
19. `[TECH] (Engine)` What mechanism should be used to propagate Dead-Ledger decision tokens from `Policy.DeadLedger.requestBundleAccess` into H.Selector and H.Director calls, so selectors never activate bundles without a fresh entitlement?
20. `[TECH] (Engine)` How should the `H.Director.applySafetyDecision` helper be wired so that it can optionally log its own decisions as telemetry events (e.g., strategy `soften`, `imply`, `refuse`) without requiring persona scripts to emit their own NDJSON?

16–20 all line up cleanly with the existing director, image-helper, Dead-Ledger, and session-route work; below are concrete shapes and wiring patterns.[30] [31] [32]

16. Annotating session metrics with director persona and comfort policy

The session metrics envelope spec already includes `aiChatHorrorProfileId` and a `comfortOutcome` field; you should extend it with explicit director and comfort policy join keys. In `schemas/telemetry/session-metrics-envelope-v1.json`, add:[31] [32]

- Top-level fields:
  - `directorPersonaId`: the active `director-persona-contract` id, or `null` if no persona governed this session.[30]
  - `directorKind`: cached from the persona contract (`coldOverseer`, `ritualStatistician`, etc.) for quick slicing.[30]
  - `comfortPolicyRef`: id of the `director-comfort-policy.v1` used when clamping DET and CDL.[31] [30]
- Per-segment or per-step fields (if your envelope is stepwise):
  - `directorStrategy`: last `H.Director.applySafetyDecision.strategy` (`allow`, `soften`, `imply`, `downshift`, `refuse`) applied before or during this step.[31] [30]
  - Optional `directorStyleHint`: tone hint returned by `applySafetyDecision` (e.g., `tone_soften`, `tone_offscreen`).[30]

H.Run must then populate these from its director envelope each time it logs segment metrics: it calls `H.Director.loadPersona` at session start, keeps `directorPersonaId` and `comfortPolicyRef` in its state, and copies them into every `session-metrics-envelope` event. Offline, you can then correlate specific comfort policies and strategy mixes (ratio of `soften` to `downshift` calls) with UEC/CDL/ARR trajectories and the envelope's `comfortOutcome` summary.[32] [31] [30]

17. Base interface for image-helper module

You already have a conceptual `H.ImageHelper.describe(context, invariants, metrics, inventory)` stub in the image-helper research; that shape should become the canonical Lua surface for AI-Chat and engines. A minimal interface:[31] [30]

```
-- Target: documented in HorrorPlace-Constellation-Contracts docs/runtime/image-hel

-- context: run & beat info only
--   runId, sessionId, regionId, tileId, nodeId,
--   eventStage (Outer, Locus, Rupture, Fallout),
--   difficulty, directorPersonaId
-- invariants: CIC, AOS, DET, SHCI, LSG, HVFmag, SPR etc. for this node/tile
-- metrics: current UEC, EMD, STCI, CDL, ARR (targets and/or actuals)
-- inventory: active interpretation tools / persona inventory for this run

function H.ImageHelper.describe(context, invariants, metrics, inventory)
  -- Returns a pure data table:
  -- {
  --   description = "text-only, implication-safe prompt",
  --   hiddenTags  = { "clue.water-rising", "symbol.ritual-glyph-broken" },
  --   metricDeltas = { UEC = 0.05, EMD = 0.1, CDL = 0.05 }
  -- }
end
```

Tier-1 repos only contain this spec and maybe a no-op or test implementation; real image generation happens in vault repos and in external tools that consume `description` and `hiddenTags`. The API contract requires:[31]

- `description` is horror-safe, implication-only, shaped by difficulty and CDL/ARR caps from the runContract; novice runs must include at least one **interpretable** symbol per image.[30] [31]
- `hiddenTags` are opaque clue IDs used by CHATDIRECTOR and telemetry to track which clues were shown vs. acted upon, never rendered directly.[31]
- `metricDeltas` give H.Metrics a suggested update to UEC/EMD/CDL/ARR if the player engages with the image, letting image-driven clues feed back into the entertainment spine.[30] [31]

Godot and Unreal plugins then just forward their current run state to this Lua function and pass the `description` out to their own image middleware; Tier-1 stays free of raw images.[31]

18. Uniform `context` table for all H. modules

To avoid parameter explosion, define a canonical `context` table shape in a public API doc (e.g., `docs/runtime/h-context-lua-api-v1.md`) and require all new H. decision functions to accept it as their first parameter. A baseline:[30] [31]

```
local ctx = {
  sessionId         = "sess-...",
```

```
  runId            = "run-...",
  regionId          = "reg-...",
  tileId           = "tile-..." ,  -- optional
  nodeId           = "node-...",  -- optional
  patternId         = "hsp-...",   -- selector pattern, optional
  directorPersonaId = "dir-...",   -- optional
  experienceContractId = "exp-...",-- optional
  stage            = "Outer",     -- run / event stage
  difficulty       = "Standard",  -- from runContract
  routerStateId     = "rtr-...",   -- tier router state
}
```

Core modules then take `context` plus any local arguments:

- `H.Selector.selectPattern(context, selectorHints)`
- `H.Node.choosenext(context)` (reads `regionId`, `nodeId`, `patternId` from `context`).
- `H.Director.applySafetyDecision(context, safetyDecision, budgetSnapshot)`
- `H.ImageHelper.describe(context, invariants, metrics, inventory)`.[30] [31]

CI in Codebase-of-Death should enforce that public H. functions either:

- Accept a single `context` table (plus module-specific extras), or
- Are low-level helpers that are not exported as part of the H. surface.[33]

That way, adding a new concern (e.g., `deadLedgerDecisionToken`) is a field addition on `context`, not a signature explosion across modules.[32]

19. Propagating Dead-Ledger decision tokens into selectors and directors

Dead-Ledger usage is already sketched for attesting dungeonRunContracts and policies; you want the decision token from `Policy.DeadLedger.requestBundleAccess` to flow into H.Selector and H.Director via the same `context` mechanism.[32] [31]

Pattern:

1. When CHATDIRECTOR or the engine needs to use a Dead-Ledger-attested bundle (runContract, event bundle, director comfort policy), it calls:

   ```
   local dlResult = Policy.DeadLedger.requestBundleAccess(deadLedgerRef, accessCon
   -- dlResult includes: { ok, decisionToken, expiresAt, scope }
   ```

2. On success, the caller attaches `deadLedgerDecisionToken = dlResult.decisionToken` to the session's `context` and caches it for the token's lifetime.[32]
3. All calls into H.Selector and H.Director that might touch attested bundles must receive that `context` and:
   - Check that `context.deadLedgerDecisionToken` is present when operating on a bundle that declares a `deadLedgerRef`.[32] [31]

- Optionally call a lightweight verifier (e.g., `Policy.DeadLedger.checkToken(decisionToken, deadLedgerRef)`) if the module wants to double-check scope/expiry.

4. If the token is missing, expired, or invalid, the selector/director must:
   - Treat the bundle as ineligible (e.g., return `NOELIGIBLEPATTERN` or `ERR_ENTITLEMENT_EXPIRED`).
   - Or downshift to non-attested, public content stacks, as defined by the age-tier router policy.[33]

This keeps Dead-Ledger decisions explicit and machine-checked: selectors never activate attested bundles without an accompanying token in `context`, and telemetry can log both `deadLedgerRef` and `decisionTokenId` to audit entitlement usage.[32] [31]

20. Wiring H.Director.applySafetyDecision for telemetry

The director safety adapter spec already defines `H.Director.applySafetyDecision(sessionId, safetyDecision, budgetSnapshot)` and recommends logging strategy verbs; the clean way is to have this helper emit its own telemetry event whenever it returns a decision.[31] [30]

Implementation-wise:

- `H.Director.applySafetyDecision(context, safetyDecision, budgetSnapshot)`:

```
function H.Director.applySafetyDecision(context, safetyDecision, budgetSnapshot)
  -- compute strategy & styleHint
  local strategy, styleHint, reason = computeStrategy(context, safetyDecision, b

  -- emit telemetry (Tier-2 implementation)
  H.Metrics.recordDirectorDecision({
    sessionId         = context.sessionId,
    runId             = context.runId,
    regionId          = context.regionId,
    directorPersonaId = context.directorPersonaId,
    comfortPolicyRef  = context.comfortPolicyRef,
    strategy          = strategy,        -- soften / imply / downshift / refuse /
    styleHint         = styleHint,       -- optional code
    safetyDecision    = safetyDecision,  -- minimal, schema-constrained view
    budgetSnapshotId  = budgetSnapshot.id or nil,
    timestamp         = os.time(),
  })

  return {
    ok = true,
    data = {
      strategy  = strategy,
      styleHint = styleHint,
      reason    = reason,
    }
  }
end
```

- `H.Metrics.recordDirectorDecision` writes an NDJSON event that conforms to a new telemetry schema such as `schemas/telemetry/director-decision-event-v1.json`, with fields for `strategy`, `directorPersonaId`, `comfortPolicyRef`, and the current entertainment metrics snapshot.[32] [31]

Persona scripts then only ever call `H.Director.applySafetyDecision`; they do not write NDJSON themselves and do not need to know the telemetry schema. CI can enforce this by forbidding direct writes to director-decision telemetry topics from persona modules and by requiring that all such events reference an `H.Director.applySafetyDecision` call site via module naming and schema checks.[33] [30] [31]

⁂

21. `[TECH] (CI)` What schema-level invariants should be added to `director-persona-contract.v1.json` and `director-comfort-policy.v1.json` to ensure that CDL spans and DET caps never widen beyond those allowed by the consent and budget modules for a given tier? [file:9]

22. `[TECH] (Engine)` How can H.Budget and H.Director share a common representation of "high DET event" versus "low DET mood" so that required cooldown ratios are enforceable in Lua without each module reinventing DET band thresholds?
23. `[TECH] (Engine)` What is the most straightforward way for a runContract (story-driven session bundle) to make its difficulty profile accessible to Lua (e.g., via `H.Run.getDifficultyBands(runId, difficultyName)`) so persona skill bands and interpretation tools can be applied consistently?
24. `[TECH] (Authoring)` How should AI-Chat request a prevalidated skeleton for a director persona or runContract, and what fields must that skeleton always contain so that AI-Chat only fills values inside schema and invariant bounds?
25. `[TECH] (Engine)` How can `H.Selector.selectPattern` and `H.Node.choose_next` share an internal scoring kernel so that pattern scoring and node scoring do not drift in behavior between pattern-level and node-level selection?

21–25 all have clean answers that plug into the existing schemas and Lua stubs without inventing new surfaces.[34] [35] [36]

21. Schema-level invariants for CDL and DET

You already sketched tier-aware constraints for DET in both `director-persona-contract.v1.json` and `director-comfort-policy.v1.json`; you want to generalize that so CDL spans also narrow, and both never exceed consent/budget ceilings for the tier.[35] [34]

In `schemas/runtime/director-persona-contract.v1.json`:

- Keep `invariantBands.DET` and `metricTargets.CDL` as they are, but add tier-specific `allOf` constraints that encode narrowing against canonical caps per tier.[34]
- For example, assuming the consent/budget module exposes tier ceilings via constants or a shared defs file, you can write:

```
"allOf": [
  {
    "if": { "properties": { "tier": { "const": "standard" } } },
    "then": {
      "properties": {
        "invariantBands": {
          "properties": {
            "DET": { "properties": { "max": { "maximum": 6.0 } } }
          }
        },
        "metricTargets": {
          "properties": {
            "CDL": {
              "properties": {
                "min": { "minimum": 0.0 },
                "max": { "maximum": 0.7 }
              }
            }
          }
        }
      }
    }
  }
]
```

- Repeat per tier (`mature`, `research`) with stricter or looser `maximum` / `minimum` values that match the horror-intensity budget and consent policies for those tiers.[35]

In `schemas/policy/director-comfort-policy.v1.json`:

- You already cap `maxDetPerSession` per tier; add a constraint tying `allowedCdlSpan` to the same CDL caps used in persona metrics.[34] [35]
- For example, for `tier: "standard"`:

```
"allOf": [
  {
    "if": { "properties": { "tier": { "const": "standard" } } },
    "then": {
      "properties": {
        "maxDetPerSession": { "maximum": 6.0 },
        "allowedCdlSpan": {
          "properties": {
            "min": { "minimum": 0.3 },
            "max": { "maximum": 0.7 }
          }
        }
      }
    }
```

```
  }
]
```

CI then enforces three levels of narrowing: persona bands inside tier caps, comfort policy further constraining DET/CDL within those bands, and runtime `H.Director.constrainMetrics` intersecting persona+policy with live consent/budget caps so CDL and DET can never widen at runtime.[35] [34]

22. Shared representation of "high DET event" vs. "low DET mood"

Rather than each module inventing its own idea of "high DET," define a shared classification schema and let both H.Budget and H.Director consume it.[36] [35]

In Constellation-Contracts, add a small policy schema, e.g. `schemas/policy/det-band-classification.v1.json`:

- Fields:
  - `tier`: `standard | mature | research`.
  - `lowDetBand`: `{ "min": number, "max": number }`
  - `mediumDetBand`: `{ "min": number, "max": number }`
  - `highDetBand`: `{ "min": number, "max": number }`
  - Optional `climaxDetBand` for research tiers.[35]

Runtime helpers:

- A tiny Lua module, e.g. `H.DetBands`:

```
function H.DetBands.classify(detValue, tier)
  -- returns "low", "medium", "high", or "climax" based on policy
end
```

H.Budget then:

- Uses `H.DetBands.classify(detDelta, tier)` when consuming a beat to tag it as `low` or `high` DET for per-run and per-session accounting, and to enforce caps like "no more than N high-DET events per window."[36] [35]

H.Director then:

- Uses the same classifier to interpret `requiredCooldownRatio` in the comfort policy: for each turn, `H.Director` knows if the last beat was `high` vs. `low` DET and can compute the rolling ratio of `low` to `high` beats.[36] [34]
- When the ratio falls below the comfort policy floor, it can veto new high-DET events via `H.Director.enforceCooldown` and request cooldown moods from selectors.[36] [35]

Because both modules read the same policy file and classifier, you do not risk drift in what counts as a high-DET event across budgeting and persona safety.[35]

23. Making runContract difficulty profiles visible to Lua

Your runContract spec already defines a `difficultyProfile` map with metric bands and skill bands per difficulty; you want a minimal runtime helper that exposes those bands to Lua without re-parsing JSON in every module.[35]

In Constellation-Contracts:

- `schemas/runtime/run-contract.v1.json` includes:

```
"difficultyProfile": {
  "type": "object",
  "additionalProperties": {
    "type": "object",
    "properties": {
      "skillBands": {
        "type": "object",
        "properties": {
          "analysisSkill": { "$ref": "defs.range01" },
          "intuitionSkill": { "$ref": "defs.range01" },
          "riskTolerance": { "$ref": "defs.range01" }
        }
      },
      "worldInvariants": {
        "type": "object",
        "properties": {
          "CIC": { "$ref": "defs.range01" },
          "AOS": { "$ref": "defs.range01" },
          "DET": { "$ref": "defs.range10" },
          "SHCI": { "$ref": "defs.range01" }
        }
      },
      "metricTargets": {
        "type": "object",
        "properties": {
          "UEC": { "$ref": "defs.range01" },
          "EMD": { "$ref": "defs.range01" },
          "STCI": { "$ref": "defs.range01" },
          "CDL": { "$ref": "defs.range01" },
          "ARR": { "$ref": "defs.range01" }
        }
      }
    },
    "required": ["skillBands", "worldInvariants", "metricTargets"]
  }
}
```

In Horror.Place runtime Lua:

- Provide a contract loader and helper:

```
-- luaruntime/hrun.lua
local Contracts = require "horrorcontracts"

local Run = {}
```

```
function Run.getDifficultyBands(runId, difficultyName)
  local runCard = Contracts.run(runId)
  if not runCard then
    return { ok = false, error = { code = "RUN_NOT_FOUND" } }
  end
  local diff = runCard.difficultyProfile and runCard.difficultyProfile[difficult
  if not diff then
    return { ok = false, error = { code = "DIFFICULTY_NOT_FOUND" } }
  end
  return {
    ok = true,
    data = {
      skillBands      = diff.skillBands,
      worldInvariants = diff.worldInvariants,
      metricTargets   = diff.metricTargets
    }
  }
end

H.Run = Run
```

Persona logic and interpretation tools then call `H.Run.getDifficultyBands(runId, difficulty)` once per session and cache the result. That gives H.Director, image-helper, and persona code a single, consistent view of difficulty-tuned skill, invariant, and metric bands for the run.[36] [35]

24. AI-Chat requesting prevalidated skeletons

You already have an AI-safe authoring session contract pattern; you can reuse that idea to add explicit "generate skeleton" APIs for persona and run contracts so AI-Chat never hand-rolls shapes.[34] [35]

In Constellation-Contracts:

- Define two small JSON "skeleton request" types:
  - `ai-authoring-director-persona-skeleton-request.v1.json` with fields:
    - `objectKind`: const `"directorPersonaContract"`.
    - `tier`: requested tier.
    - `directorKind`: one of the allowed persona kinds.
    - Optional `experiencePreset`: link to an experienceContract profile, if you want default bands.[35]
  - `ai-authoring-run-contract-skeleton-request.v1.json` with fields:
    - `objectKind`: const `"runContract"`.
    - `tier` and `difficultyNames` (e.g., `["Novice","Standard"]`).
    - Optional `experienceMode` for template selection.[35]

In the authoring pipeline:

- AI-Chat first submits a session planning envelope that declares it will request one of these skeletons.[34] [35]
- A dedicated CLI/HTTP tool in Constellation-Contracts or Orchestrator, e.g. `chatdirector-generate-skeleton`, takes the request and returns a JSON skeleton that already:
  - Has `id`, `schemaRef`, `objectKind`, `tier`, and `comfortPolicyRef`/`difficultyProfile` filled with safe default bands that narrow the tier caps.[34]
  - Ensures `invariantBands` and `metricTargets` are inside spine and consent/budget bands via `validateinvariants.rs`.[35]

AI-Chat then only edits allowed fields:

- Names, tone labels, tags, notes for personas.
- `eventSequence`, `moodCurves`, and descriptive text fields for runs.

CI enforces that AI-Chat cannot widen numeric ranges: if a skeleton says CDL `min: 0.4, max: 0.7`, any attempt to push `max` above 0.7 fails validation. That keeps AI-Chat authoring inside prevalidated envelopes.[34] [35]

25. Shared scoring kernel for patterns and nodes

You already have `H.Selector.selectPattern` at pattern level and `H.Node.scorenextfromhistory` at node level; they both use liminal tags, mechanic hooks, and SHCI but currently implement scoring separately. To avoid drift, factor the scoring logic into a shared Lua module and have both call into it.[34]

In Horror.Place runtime Lua:

- Create `enginelibrary/horror_scoring.lua`:

```
-- enginelibrary/horror_scoring.lua
local Scoring = {}

-- patternConfig: history-selector-pattern config
-- candidate: table with fields from contracts/invariants, e.g.
--   liminalTags, mechanicHooks, SHCI, CIC, AOS, etc.
function Scoring.scoreCandidate(patternConfig, candidate)
  local score = 0.0

  -- Liminal bias
  local liminalPrefs = patternConfig.liminalTagPreferences or {}
  local liminalTags  = candidate.liminalTags or {}
  -- ... compute liminal contribution using weights and tags ...

  -- Mechanic hook bias
  local hookPrefs = patternConfig.mechanicHookPreferences or {}
  local hooks     = candidate.mechanicHooks or {}
  -- ... compute mechanic contribution ...

  -- History coupling (SHCI)
  local hc   = patternConfig.historyCoupling or {}
  local shci = candidate.SHCI or 0.0
```

```
  -- ... apply minSHCI, preferredBand, inside/outside weights ...

  -- Optional entertainment targets / metric alignment
  local et = patternConfig.entertainmentTargets
  if et and candidate.metrics then
    -- e.g. reward being inside UEC/CDL bands
    -- ... compute contribution ...
  end

  return score
end

return Scoring
```

Then:

- `H.Selector.selectPattern` uses `Scoring.scoreCandidate` to score region-level or pattern-level candidates, passing each candidate's liminal, mechanic, SHCI and metric snapshot.[34] [35]
- `H.Node.scorenextfromhistory` already builds a `candidateCard` per neighbor; you adapt it to build a `candidate` table and pass it to `Scoring.scoreCandidate(patternConfig, candidate)` instead of having its own local `scoreCandidateNode` function.[34]

By centralizing the scoring rules:

- Any tuning of liminal preference, mechanic biases, or SHCI bands happens in one module; both pattern selection and node traversal respond identically to those numbers.[35] [34]
- CI can add a simple test suite that feeds a few synthetic candidates into `Scoring.scoreCandidate` and asserts expected ordering; selectors and nodes will both inherit those guarantees.[34]

⁂

26. `[TECH] (Telemetry)` What additional fields should a `history-selector-decision` telemetry event carry (e.g., `patternId`, `nodeId`, `directorPersonaId`, metric snapshot) to reconstruct full decision trees for playtest analysis?

27. `[TECH] (Engine)` How can AI-Chat and engines agree on a canonical `stage` enumeration for runs (e.g., OuterThreshold, Locus, Rupture, Fallout) so templates, mood contracts, and selectors all talk about progression using the same vocabulary?
28. `[TECH] (Engine)` What interface should `H.Run` expose (e.g., `start`, `step`, `currentStage`, `isComplete`) so that directors, selectors, and AI-Chat templates know exactly where they are in a runContract without re-deriving state from metrics alone?
29. `[TECH] (Authoring)` How can we define a small markdown-to-metadata contract for design docs so that AI-Chat can parse which invariants and metrics a doc references and use that information to shape its authoring behavior?

30. `[TECH] (Engine)` What is the simplest way to provide a `H.Debug.logDecision(context, details)` helper that writes to structured logs, and how can we restrict its use so it never leaks sensitive or vault-only IDs into public telemetry?

26–30 all slot cleanly into the existing selector, run, telemetry, and debug surfaces; below is how I'd harden each in Constellation-Contracts and Horror.Place.[37] [38] [39]

26. history-selector-decision telemetry fields

You already log `selectorPatternId`, rule snapshots, and metric snapshots for history selector decisions; to reconstruct full decision trees you need all the join keys plus local structural context.[38] [37]

The `history-selector-decision-event-v1.json` payload should always carry:

- Session and run keys: `sessionId`, `runContractId`, `dungeonRunId`, `regionId`, `dungeonId`, `mapStyle`, and `stage` (canon enum, see 27) so decisions can be threaded along a single run and across runs.[39] [37]
- Selector context: `selectorPatternId`, `selectorRuleSetId`, and a `ruleCatalogSnapshot` listing each rule ID, phase, action, and whether it applied, as in the Lua stub.[38]
- Node and traversal context (when applicable): `currentNodeId`, `nextNodeId`, `nodeRole`, `choseLiminalTaggedNode`, `liminalTags`, `mechanicHooks`, and any per-node SHCI/LSG snapshot you use to steer traversal.[37] [38]
- Metrics before/after: `metricsBefore` and `metricsAfter` subobjects with UEC, EMD, STCI, CDL, ARR plus derived deltas, so you can attribute movement toward or away from run targets to particular decisions.[39] [37]
- Director and persona: `directorPersonaId`, `directorStrategy` (allow/soften/imply/refuse/downshift), and `comfortPolicyId`, to analyze how different persona strategies interact with selector choices.[38] [39]
- Budget and safety joins: `budgetEventId`, `safetyDecisionId`, and a `resolutionStrategy` field (`normal`, `noEligiblePattern`, `profileCap`, etc.) so you can see which decisions were constrained or rerouted.[39] [38]

With that shape, you can reconstruct decision trees by grouping on `sessionId` and ordering by timestamp, then branching on `currentNodeId → nextNodeId` while tying each branch to its pattern, director, and metric deltas.[37] [38]

27. Canonical run stage enumeration

The route envelope research already assumes named stages like OuterThreshold, Rupture, and Fallout. To keep engines and AI-Chat in sync:[37] [39]

- Define a small shared schema `schemas/runtime/run-stage-enum.v1.json` in Constellation-Contracts with the canonical set, e.g. `OuterThreshold`, `Locus`, `Rupture`, `Fallout`, plus any intermediate phases you need.[39]
- Reference this enum in:

    - `run-contract.v1.json` (`allowedStages` per run, and optionally a `defaultStageCurve` mapping step indices to stages).[39]
    - `ai-chat-template-contract.v1.json` (a `validStages` array so templates declare where in the curve they apply).[39]
    - `history-selector-pattern-v1.json` (an optional `stageAffinity` block so patterns can prefer or avoid certain stages).[38] [39]
    - Telemetry route envelopes and `history-selector-decision` events (`stage` on each step).[37] [39]
- In Horror.Place, have `H.Run` own the authoritative `currentStage` for a session, derived from the runContract's stage curve and progression rules, not from metrics. AI-Chat and selectors then read `H.Run.currentStage(sessionId)` rather than inferring stage from DET/UEC alone, so everyone speaks the same stage vocabulary.[37] [39]

28. H.Run interface shape

You already sketched a run state machine and a Godot bridge; the public runtime surface can be narrowed to a small, canonical set of calls that hide state derivation.[37] [39]

At minimum in Horror.Place's Lua runtime:

- `H.Run.start(runContractId, personaContractId, sessionId) → result`
  Loads the run and persona contracts, validates them against schemas and invariants, binds consent and budget, and returns a small descriptor including `runId`, initial `stage`, and allowed difficulties.[39]
- `H.Run.step(sessionId, userInput, engineContext) → result`
  Core per-turn function that:
    - Applies selector and director decisions.
    - Advances internal stage according to the run's curve and rules.
    - Emits updated `stage`, `metricsSnapshot`, and any image helper payload.[37] [39]
- `H.Run.currentStage(sessionId) → stageName`
  Returns the current canonical stage label, so directors, templates, and selectors don't re-derive it from metrics.[39]
- `H.Run.isComplete(sessionId) → boolean`
  Indicates whether the run has reached a terminal stage (e.g., completed Fallout) under runContract rules, independent of whether the chat session continues for meta discussion.[39]
- Optional helpers: `H.Run.getDifficultyBands(runId, difficultyName)` (as in your earlier question), and `H.Run.getProgress(sessionId)` returning normalized progress through the run (0–1) and last few stages visited.[37] [39]

All of these should return the standard `ok/data/error` envelope, and they should be the only way other modules learn about run progression to keep stage semantics and completion logic centralized.[39]

29. Markdown-to-metadata contract for design docs

You want AI-Chat to understand which invariants and metrics a design doc touches so it can stay inside those bands while authoring. The simplest pattern is a small, standardized front-matter block plus a schema to validate it.[38] [39]

In Constellation-Contracts:

- Define `schemas/runtime/design-doc-metadata.v1.json` with fields such as:
  - `docKind`: e.g. `dungeonDesign`, `personaDesign`, `runDesign`.
  - `targetRepos`: list of repo IDs (Horror.Place, Codebase-of-Death, etc.).
  - `invariantsUsed`: subset of `CIC, MDI, AOS, RRM, FCF, SPR, RWF, DET, HVF, LSG, SHCI`.[38]
  - `metricsUsed`: subset of `UEC, EMD, STCI, CDL, ARR`.[38] [39]
  - `modulesTouched`: subset of `H.Selector`, `H.Node`, `H.Run`, `H.Director`, etc.
  - Optional `intendedTiers` and `experienceModes` (for run and persona docs).[39]
- Authoring docs then start with a fenced YAML block or similar header that matches this schema, which CI validates. For example:[39]

```
---
docKind: dungeonDesign
targetRepos: [Horror.Place, HorrorPlace-Codebase-of-Death]
invariantsUsed: [CIC, DET, LSG, SHCI]
metricsUsed: [UEC, EMD, CDL, ARR]
modulesTouched: [H.Selector, H.Node, H.Run]
intendedTiers: [standard, mature]
---
```

AI-Chat tooling:

- When a designer pastes or opens a doc, AI-Chat first parses this header and treats it as an AI-safe authoring profile for the session.[38] [39]
- CHATDIRECTOR compares the header to the session's ai-authoring contract and refuses edits that would introduce invariants or metrics outside the declared set or widen bands beyond profile caps.[39]
- AI-Chat uses the header to bias its suggestions: if a doc only references DET, SHCI, and CDL, it should not invent tuning advice for HVF or ARR, and any generated skeletons or code should only touch those invariants.[38] [39]

This keeps design docs machine-readable without changing how they're written narratively.

30. H.Debug.logDecision helper and leakage controls

You want a simple debug helper that writes structured logs without leaking vault-only IDs into public telemetry. The pattern is:[37] [39]

- Two layers of logging:
  - `H.Debug.logDecision(context, details)` for local development and CI logs, which may include internal IDs but never ship to production telemetry sinks.[39]
  - `H.Telemetry.emitDecision(event)` for production NDJSON, which enforces a whitelist of allowed fields and ID patterns and strips or hashes anything else.[37] [39]

In Horror.Place:

- Implement `H.Debug.logDecision` as a thin wrapper around a logger that is only active in debug builds or when a specific flag is set:

```
function H.Debug.logDecision(context, details)
  if not H.Debug.enabled then return end
  -- context: { module = "H.Selector", sessionId = ..., runId = ... }
  -- details: small table of debug info (scores, candidates, internal IDs)
  -- This writes to an engine-local log or dev-only file, never NDJSON telemetry
end
```

- In `H.Selector.recordDecision`, call `H.Debug.logDecision` with full internals, but construct a separate sanitized `history-selector-decision` payload that only includes fields allowed by the telemetry schema (`sessionId`, `runId`, `patternId`, `nodeId`, metrics, etc.).[38] [39]

To enforce non-leakage:

- Constellation-Contracts defines a telemetry schema with strict field sets and ID regexes; any field not in the schema is rejected by CI and at runtime.[39]
- Telemetry emission helpers (`H.Telemetry.emitDecision`, `H.Metrics.recordEvent`, etc.) only accept structured payloads that validate against these schemas and never accept arbitrary debug blobs.[37] [39]
- A linter in Codebase-of-Death ensures `H.Debug.logDecision` is never called in code paths marked as production (e.g., behind a `DEBUG` flag or environment check), and that telemetry modules never import `H.Debug`.[39]

That keeps rich decision traces available for developers while ensuring anything that leaves the engine is governed by the telemetry schemas and cannot expose vault-only IDs or raw internal structures.

⁂

31. `[TECH] (CI)` How should CI enforce “three artifacts per request per repo” in a way that is visible to AI-Chat, so chat-based authoring tools know when they must split work into multiple phases rather than overfilling a single prism envelope?

32. `[TECH] (Engine)` How can we standardize the representation of “persona inventory” (list of interpretation tools) in Lua so both image-helper and director persona logic can reason about which clues and contradictions are allowed this run? [file:9]
33. `[TECH] (Engine)` What data structure should be used for representing node graphs (adjacency lists vs. explicit edge objects) so H.Node functions can easily apply pattern-based scoring without heavy transformations at runtime?
34. `[TECH] (Engine)` How can Death-Engine expose H. functions to Blueprint or VisualScript-like systems so designers can easily wire selector and director calls without writing Lua by hand, while still keeping Lua as the single behavior authority?
35. `[TECH] (Authoring)` What small JSON manifest should each repo maintain listing which H.* namespaces it expects to use (e.g., `H.Audio`, `H.Selector`, `H.Director`), so AI-Chat knows which runtime surfaces are appropriate when generating code for that repo?

31–35 all map cleanly onto the existing CHATDIRECTOR, H.* runtime, and CI doctrine; below is how I’d wire each so AI-Chat and engines can actually rely on them.[40] [41] [42]

31. CI-visible “three artifacts per request per repo”

The easiest way to enforce this is to make the limit part of the authoring envelope and to have both CI and CHATDIRECTOR read the same cap.[42]

In Constellation-Contracts:

- Extend the existing ai-authoring envelope (chatDirector request/response) with a small field such as `perRepoArtifactCaps`, mapping repo IDs to integer caps (default 3). For example, the envelope might declare `Horror.Place: 3`, `HorrorPlace-Codebase-of-Death: 3` for a given request.[42]
- Have CHATDIRECTOR track, per envelope, how many artifacts have already been emitted to each repo during this request, and include the current counts and remaining budget in every `ai-authoring-response` header.[42]

In CI (HorrorPlace-Orchestrator):

- A reusable workflow scans the diff for each PR, infers which files belong to which authoring envelope (via `prismMeta` or a dedicated `authoringRequestId`), and asserts that no repo has more than three new or modified artifacts for that envelope.[42]
- If AI-Chat tries to emit a fourth file for `Horror.Place` under the same `authoringRequestId`, CI fails with a structured error like `ERR_ARTIFACT_CAP_EXCEEDED`, including `maxPerRepo: 3` and `currentCount`.[42]

Visibility to AI-Chat:

- CHATDIRECTOR mirrors the CI rule by returning this error immediately once the cap is hit and by including a `remainingArtifactsByRepo` map in each response, so templates can explicitly tell coding-agents "this run has exhausted Horror.Place; start a new authoring phase for additional files." [40] [42]

32. Standard "persona inventory" representation in Lua

Your design questions already call out "interpretation tools" and how they modulate UEC/EMD/CDL, so the inventory should be a small, shared table shape that both H.ImageHelper and H.Director can read.[40] [42]

In Constellation-Contracts:

- Extend `director-persona-contract.v1.json` with a `inventory` array of tool descriptors, each with at least `{ toolId, kind, metricEffects }`, where `metricEffects` declares deltas or constraints on UEC, EMD, STCI, CDL, ARR when the tool is active.[40] [42]

In Horror.Place Lua:

- Normalize persona inventory into a canonical runtime table when loading a persona, e.g.:

```
persona.inventory = {
  -- Each entry is a small, stable record
  ["pattern-recognition-lens"] = {
    id = "pattern-recognition-lens",
    kind = "interpretation",
    metrics = { UEC = +0.05, EMD = +0.10, CDL = -0.05, ARR = +0.05 },
  },
  ["doubt-amplifier"] = {
    id = "doubt-amplifier",
    kind = "interpretation",
    metrics = { UEC = +0.10, EMD = +0.05, CDL = +0.10, ARR = +0.10 },
  },
}
```

- Expose a tiny helper namespace, e.g. `H.PersonaInventory`, with functions like:
  - `H.PersonaInventory.activeTools(sessionId) → list`
  - `H.PersonaInventory.aggregateMetricEffects(sessionId) → metricDeltas`

H.ImageHelper and H.Director then consume this shared representation: image-helper can bias prompts toward more/less explicit clues based on active tools, and director logic can adjust how many contradictions it introduces without redefining inventory semantics per module.[40] [42]

33. Node graph representation for H.Node

The existing `enginelibrary/horrornodes.lua` already assumes adjacency lists via `topology.connections` with `targetNodeId` and extra flags. That's a good fit; you don't need a heavier edge object graph for v1.[40]

Recommended approach:

- Keep node contracts as node-centric records with a `topology.connections` array, each entry being a small edge descriptor `{ targetNodeId, kind, hidden?, oneway?, tags? }`. [41] [40]
- At load time, build an in-memory adjacency map keyed by `nodeId`:

```
dungeonGraph[nodeId] = {
  card = nodeCard,
  neighbors = { -- copy of connections with resolved metadata if needed
    { targetId = "node-stairwell-02", kind = "standard", hidden = false },
    { targetId = "node-maintenance-03", kind = "sidebranch", hidden = true },
  },
}
```

- H.Node helpers (`describe`, `scorenextfromhistory`, `choosenext`) then just iterate `neighbors` for the current node and call `scoreCandidateNode` per neighbor; there's no extra transformation step at runtime.[40]
- Explicit edge objects become useful only if you start modeling edge-level invariants or mechanics; until then, adjacency lists plus edge flags (`hidden`, `ritualLocked`, etc.) are enough and keep scoring fast.[41] [40]

This matches your current helper, which walks `currentCard.topology.connections` directly and scores each neighbor using invariants and pattern preferences.[40]

34. Exposing H.* to Blueprint / VisualScript in Death-Engine

You already standardize H.* as the Lua orchestration layer; in Death-Engine the goal is to wrap that in a very thin, nodes-friendly facade while keeping Lua authoritative.[42]

Pattern:

- In Codebase-of-Death, implement a small C++/Rust bridge that:
  - Embeds the Lua runtime and loads H.* modules (`horror-engine-lua-api-v1.lua` plus `horrornodes.lua`, selector wrappers, etc.).[42] [40]
  - Exposes a narrow set of engine-facing C-ABI functions or bindings, e.g. `Call_H_Run_Step`, `Call_H_Selector_SelectPattern`, `Call_H_Node_Describe`, each taking simple POD structs or JSON blobs.[42]
- In Blueprint/VisualScript:
  - Create custom nodes that map 1:1 to these bridge functions:
    - `HRun_Start`, `HRun_Step` (wrapping `H.Run.start`/`H.Run.step`)

    - `HSelector_SelectPattern`
    - `HNode_Describe`, `HNode_ChooseNext`[40] [42]
  - These nodes only marshal arguments and results; all behavior remains in Lua. Designers wire these nodes in graphs to drive encounters, but they never call low-level invariants or selectors directly.[42]

Governance:

- Codebase-of-Death CI ensures that the bridge only exposes functions that exist in the official H. API and that no Blueprint node bypasses H.Safety, H.Budget, or Policy.DeadLedger.[42]
- Any new H.* namespace must be registered in this bridge before Blueprint can see it, so design graphs cannot circumvent the orchestration layer.[42]

35. Repo manifest listing expected H.* namespaces

You already use repo manifests to route objectKinds and tiers; extend that to cover runtime surfaces so AI-Chat knows which H.* namespaces are legal in a given repo.[42]

In Constellation-Contracts:

- Add or extend `repo-manifest.hpc.<repo>.json` with a field like `allowedHNamespaces`, a simple array of strings:

```
{
  "repoId": "HorrorPlace-Codebase-of-Death",
  "allowedHNamespaces": [
    "H.Consent",
    "H.Budget",
    "H.Safety",
    "H.Selector",
    "H.Node",
    "H.Run"
  ]
}
```

- CI validates that each repo manifest declares this field and that any Lua file under that repo only imports or calls H.* modules from this whitelist.[42]

AI-Chat usage:

- CHATDIRECTOR includes the manifest's `allowedHNamespaces` in the authoring envelope it passes to AI-Chat or coding-agents for that repo.[42]
- Prompt tooling and linters can then refuse to generate calls to `H.Audio` or `H.ImageHelper` if they're not on the list, and can encourage use of approved surfaces (e.g., `H.Selector` and `H.Node` in Codebase-of-Death, but not `H.Director` if that's meant to live only in Horror.Place).[40] [42]

This keeps runtime wiring predictable and ensures cross-repo code generation never introduces calls to modules the local engine adapter doesn't support.

⁂

36. `[TECH] (CI)` How can we build a linter that checks that any AI-Chat-generated Lua file only imports approved modules (H.*, engine adapters, Dead-Ledger client), and never uses forbidden standard library calls (e.g., filesystem, networking) in constellated repos?

37. `[TECH] (Engine)` How should we model and store cross-session lore progress (e.g., completed lore threads) so that selectors can favor deeper threads over time, and how can this state be exposed to AI-Chat without leaking user-level PII?
38. `[TECH] (Engine)` What is the best representation for a "comfort outcome" summary (`withinBand`, `tooIntense`, `tooFlat`) at the end of a session, and how should H.Director and H.Metrics compute it from CDL, ARR, consent wave, and budget overages? [file:9]
39. `[TECH] (Engine)` How can we extend the AI-safe authoring contract so each session planning envelope explicitly declares which H.* namespaces it will use, and how can CHATDIRECTOR refuse plans that attempt to touch any not whitelisted for that profile?
40. `[TECH] (Telemetry)` What small set of derived metrics (e.g., max DET overshoot, average UEC deviation from target, count of safety-triggered downshifts) should each session emit, and how can this be standardized across all experience types and director personas?

Answers below stay aligned with the existing spine (H.* Lua API, selector/telemetry contracts) and can be dropped into the current constellation without fighting any of the prior work.[43] [44] [45]

36. CI linter for Lua imports and forbidden stdlib

The linter should be a shared CI job that runs over all Lua in constellated repos and enforces a simple policy: only approved modules may be `require`d, and forbidden stdlib functions are never referenced.[45]

The policy source of truth should live in HorrorPlace-Constellation-Contracts as a small JSON spec, for example `schemas/runtime/lua-sandbox-policy.v1.json`, declaring per-repo or per-tier lists of allowed `require` targets (`H.*`, engine adapters, Dead-Ledger client) and forbidden globals (`io`, `os.execute`, `socket`, raw `require` outside the whitelist). A reusable GitHub Actions workflow in HorrorPlace-Orchestrator (for example `.github/workflows/lua-sandbox-validate.reusable.yml`) runs a custom linter over changed `.lua` files, parses ASTs to extract all `require(...)` targets and global identifier uses, and fails any PR that violates the policy, returning structured error codes like `LUA_IMPORT_FORBIDDEN` and `LUA_FORBIDDEN_STD_CALL` keyed by the sandbox policy. AI-Chat-generated files do not need a

special path: they are simply part of the repo, so the same linter automatically constrains them to the approved H.* and adapter surfaces.[45]

37. Cross-session lore progress storage and exposure

Lore progress should be modeled exactly like the cross-session selector state the history-selector rule schema already sketched: a small, persistent "lore progress" envelope keyed by an opaque user or profile ID, not by PII. A good pattern is the `crossSessionStateContract` block shown for selector rules, where lore threads are tracked as objects keyed by `PAT-LORE-*` IDs with fields like `completedBeats`, `unlockedTags`, and optional structural weights. This state lives in a vault repo (e.g., HorrorPlace-Black-Archivum) and is referenced by session route envelopes via an opaque `userId` and `loreThreadProgress` keys, but never contains names, emails, or free-text narrative; selectors see only IDs and counts, which lets them bias toward deeper or newly unlocked lore threads over time while keeping user PII outside the runtime history layer.[45]

To expose this state to AI-Chat safely, CHATDIRECTOR should project it into the session route envelope and selector context as simple numeric and tag summaries such as "lore thread X completed" or "maturelore unlocked," along with aggregate measures like ARR bands and mystery completion ratios. That way, chat templates and planning personas can favor deeper threads, but they only see abstract lore-thread IDs and tags (e.g., `depth-2`, `maturelore`), never any user identifier or raw lore text tied to a person.[43] [45]

38. Comfort outcome summary representation and computation

The comfort outcome fits naturally into the existing session route envelope as a small enum field, for example `comfortOutcome` with values `withinBand`, `tooIntense`, `tooFlat`, as already sketched in the telemetry design. The representation should sit alongside `metricsSummary` in the route envelope and be computed by H.Metrics and H.Director at session end using CDL, ARR, consent waves, and budget usage: H.Metrics first aggregates metric trajectories over the run (average CDL, variance, time spent outside target bands, ARR relative to contract targets), while H.Director compares those aggregates against the persona's comfort policy and consent/budget caps to decide whether intensity remained in its intended band, exceeded DET/CDL or budget ceilings (tooIntense), or systematically undershot target DET/EMD/CDL ranges (tooFlat).[45]

Consent waves and budget overages become explicit inputs to this decision: if the session triggered repeated downshifts or hard guardrail activations due to high DET/CDL or SAF violations, H.Director should bias toward `tooIntense`, while sessions that never reach the lower bounds of DET or CDL targets in the runContract should be marked `tooFlat` even if they remained safe. The aim is that each route envelope captures not just raw metric curves but a single, machine-readable comfort verdict that can be correlated with persona, pattern, and run contracts in offline analysis.[45]

39. Session planning envelopes and H.* namespace whitelists

The AI-safe authoring contract already treats planning envelopes as the root artifact for AI-Chat; extending it to include an explicit H.* namespace whitelist is straightforward and keeps CHATDIRECTOR in control. In HorrorPlace-Constellation-Contracts, the planning schema (for example, `schemas/runtime/ai-safe-authoring-contract.v1.json`) should gain a field like `allowedHNamespaces`, an array of strings such as `H.Audio`, `H.Selector`, `H.Director`, `H.Dungeon`, `Policy.DeadLedger`, keyed to the repo and tier. CHATDIRECTOR then passes this whitelist into each AI-authoring session envelope, and any plan or code snippet proposed by AI-Chat that attempts to call `H.*` namespaces not in the list is rejected at planning time with a structured error (for example, `H_NAMESPACE_NOT_ALLOWED`) before files are written.[45]

On the engine side, the same whitelist should be visible to the Lua sandbox policy and runtime linter from question 36, so Lua files are statically constrained to those namespaces, and any attempt to add new H.* dependencies requires updating the planning envelope schema and repo manifest. This keeps Lua as the single behavior authority but makes AI-Chat and other authoring tools explicitly declare which parts of the runtime surface they intend to use per session and per repo, with CHATDIRECTOR acting as the gatekeeper.[43] [45]

40. Standard derived metrics per session

The telemetry module already describes session route envelopes and history-selector decisions; a small set of derived metrics can be layered on top across all experience types and personas. Each session route envelope should emit at least: (1) maximum DET overshoot, defined as the largest positive difference between observed DET and the tighter of consent and runContract DET maxima over the session; (2) average absolute deviation of UEC from its target band center, quantifying how well uncertainty engagement stayed inside design bounds; and (3) counts of safety-triggered downshifts and softens, derived from H.Safety and H.Director decisions recorded along the route.[43] [45]

Because route envelopes already track which director persona, comfort policy, and run/dungeon contracts were active, these derived metrics can be standardized as a small `derivedMetrics` block with consistent field names across all experiences: for example, `maxDetOvershoot`, `avgUecDeviation`, `safetySoftens`, `safetyDownshifts`, and maybe `arrBandHitRate` for the fraction of steps where ARR stayed inside its target band. By keeping the shapes identical regardless of experience type or persona, offline analysis can compare comfort performance and pacing quality across modes and directors without having to special-case each run style.[43] [45]

⁂

2. Design Methodology & Metric Calibration (41–70)

41. `[DESIGN] (Design)` How should we numerically distinguish a "Slowburn Dread" experience from a "Short Shock" experience in terms of UEC, EMD, STCI, CDL, and ARR bands, and what default runContracts should ship for each as reference implementations?
42. `[DESIGN] (Design)` How can we define a small set of named experience contracts (e.g., Guided Ritual Investigation, Process-Driven Catastrophe Run) whose metric bands and invariant envelopes can be easily communicated to users as sliders or presets in an AI-Chat UI? [file:9]
43. `[DESIGN] (Design)` What numeric patterns in SHCI, CIC, and AOS should indicate that a region is suitable for "overseer-style" director personas, and how do we keep those regions rare enough that these personas feel special rather than ubiquitous? [file:9]
44. `[DESIGN] (Design)` How should we calibrate DET bands for different tiers so that "standard" sessions feel tense but not overwhelming, while "mature" and "research" tiers can support higher peaks without causing runaway distress?
45. `[DESIGN] (Design)` What guidelines should we give to narrative designers for setting ARR bands so endings feel meaningfully ambiguous but not cheap, and how can we use ARR and CDL together to distinguish satisfying ambiguity from simple confusion?

Here is a numeric design pass for 41–45 that fits the existing metric ranges, runContract shape, and director persona envelopes.[46] [47]

41. Slowburn Dread vs Short Shock

A **Slowburn** run keeps uncertainty and mystery high, spreads contrast over time, and avoids big CDL spikes; a **Short Shock** run spends DET/STCI quickly with sharper CDL spikes and lower ARR at the end.[47]

You can encode the difference in `metricTargets` bands on two reference `runContract` templates (numbers are 0.0–1.0 bands, except DET which is handled by comfort policies and events):

- Slowburn Dread (20–40 turns, "novice/standard" default):
  - UEC: $0.6–0.85$ (high sustained engagement).[47]
  - EMD: $0.3–0.6$ (steady clue density but not overwhelming).[47]
  - STCI: $0.4–0.7$ (clear safe/threat contrast, but with ramps).[47]
  - CDL: $0.3–0.6$ (moderate dissonance; enough to feel weird, not incoherent).[47]
  - ARR: $0.7–1.0$ (many mysteries unresolved or partially resolved).[46] [47]
- Short Shock (8–16 turns, "standard/mature" default):
  - UEC: $0.5–0.75$ (fast engagement, but not as long-sustained).[47]

- EMD: 0.2–0.4 (fewer distinct mysteries, focus on the main hit).[47]
- STCI: 0.6–0.9 (strong swings between safety and threat).[47]
- CDL: 0.5–0.8 (short bursts of high dissonance).[47]
- ARR: 0.4–0.7 (more mysteries resolved or collapsed by the end).[47]

A default Slowburn reference `dungeonRunContract` can mirror the mansion lobby example but with conservative DET caps and the bands above for `metricTargets`, while a Short Shock reference can be a smaller NODEGRID or WIREAUTOMAP run with higher STCI/CDL bands and a lower ARR upper bound, both shipped as Tier1 templates under `contracts/run/` or `contracts/dungeon/`.[46] [47]

42. Named experience contracts as user presets

Experience contracts like **Guided Ritual Investigation** and **Process-Driven Catastrophe Run** should just be thin JSON "profiles" that bundle metric bands, invariant envelopes, director persona, and difficulty into a preset CHATDIRECTOR can advertise as sliders.[47]

Each `experienceContract` can declare:

- A **descriptor**: name, short description, and friendly tags like "investigation-heavy", "process horror", mapped directly to UEC/EMD/STCI/CDL/ARR presets.[47]
- Metric bands and tier: for example, Guided Ritual Investigation might use UEC 0.5–0.7, EMD 0.2–0.3, CDL 0.4–0.6, ARR 0.7–0.9, while Process-Driven Catastrophe Run raises EMD and CDL but keeps ARR mid-high so failures still feel interpretable rather than random.[47]
- A small invariant envelope for the run: typical CIC/AOS/DET/SHCI ranges, which the router maps onto region and runContracts, plus a reference to a director persona whose `metricTargets` sit inside those bands.[46] [47]

In UI, the experience contract just surfaces as 3–5 labeled presets (e.g., "Slowburn Ritual", "Catastrophe Run"), each internally selecting one of these JSON profiles; CHATDIRECTOR then pins `runContract.metricTargets` and persona envelopes to those profiles and rejects per-session overrides that would escape their bands.[47]

43. Overseer-style director regions (SHCI, CIC, AOS)

Overseer personas should only run in regions whose invariants signal dense, coherent spectral history: high CIC and SHCI, with sufficiently high but not totally opaque AOS.[47]

A simple numeric pattern that marks a region as suitable:

- CIC: midpoint around 0.6–0.8 with bands not dipping below about 0.5 (strong catastrophic imprint).[46] [47]
- AOS: midpoint around 0.5–0.8 (archives are thick or contradictory but not so opaque that nothing can be inferred).[46] [47]

- SHCI: band $\geq 0.6–1.0$ (spectral-history coupling is strong; local entities are tightly tied to documented events).[46] [47]

To keep these regions rare, the world registry should limit the fraction of regionContractCards with `SHCI.min ≥ 0.6` and `CIC.min ≥ 0.5` to a small percentage of the atlas, and selector patterns that require overseer personas should explicitly check these thresholds before allowing a persona like Cold Overseer to be bound. In practice, you can enforce a rule in registry CI that only certain curated regions carry the "overseer-eligible" tag and invariant envelope, and CHATDIRECTOR refuses overseer personas elsewhere, keeping them special.[47]

44. DET bands for standard vs mature vs research tiers

DET is already treated as a scalar that comfort policies and persona envelopes clamp per tier, with Tier "standard", "mature", and "research" mapping to increasing per-session and per-event caps. A practical calibration:[47]

- **Standard** tier (public/consumer):
  - Per-event DET band for personas: roughly $1.0–4.0$.[47]
  - Max cumulative DET per session (comfort policy) around $4.0–6.0$, with required cooldown ratios enforcing at least a 2:1 or 3:1 ratio of low-DET to high-DET segments.[47]
  - CDL floors and allowed spans tuned so confusion never stays high for long, keeping sessions tense but not overwhelming.[47]
- **Mature** tier:
  - Per-event DET band extended to about $2.0–6.0$, with personas allowed sharper peaks.[47]
  - Max cumulative DET per session around $6.0–8.0$, with slightly looser cooldown ratios (for example, 1.5:1 low/high DET) to permit denser sequences.[47]
  - CDL span widened modestly, allowing more sustained dissonance but still bounded by content boundaries and ethics guardrails.[47]
- **Research** tier:
  - Per-event DET up to the global max (for example $8.0–10.0$), and per-session DET up to tier caps, but always intersected with consent, budget, and comfort policies.[47]
  - Cooldown ratios can relax, but guardrails (e.g., SAF-015 reserve floors) still enforce hard limits on runaway distress.[47]

Persona schemas and comfort policies should encode these ceilings explicitly by tier, and H.Director/H.Metrics should always intersect intended DET with the strictest of persona band, consent caps, and comfort policy, so "standard" remains tense but bounded while "mature"/"research" can explore higher peaks under controlled conditions.[47]

45. ARR bands and endings: satisfying ambiguity vs confusion

ARR is already defined as a weighted fraction of unresolved mysteries, with unresolved and partially resolved instances weighted more heavily at structurally important nodes. For endings to feel "meaningfully ambiguous", designers should aim for ARR bands that leave a substantial fraction of mysteries unresolved but keep CDL within a moderate span.[47]

Guidelines:

- In most **standard** runs, ending ARR bands around 0.6–0.85 tend to feel like "there's still more to uncover" rather than "nothing was explained", especially if CDL ends in a mid-range band such as 0.4–0.6.[46] [47]
- If ARR is high (> 0.7) but CDL is also high (> 0.7) at session end, players are likely to experience simple confusion: too many threads unresolved and too much dissonance.[47]
- For "satisfying ambiguity", set ARR high but ensure comfort policies and runContracts constrain end-state CDL to a moderate band; that usually means a few anchor mysteries resolved (or clearly signposted) while others remain open.[47]

Narrative guidelines can reflect this numerically: for each planned ending, designers should (a) enumerate core mysteries and mark which are intended to resolve, partially resolve, or stay unresolved, and (b) choose ARR bands that match that plan while using CDL bands to cap how many contradictory clues remain in play at the final node. Tooling can then warn when an ending's ARR and CDL combination falls into the "cheap" zone (high ARR, high CDL, few structural resolutions), nudging designers toward adding one or two clarifying beats that lower CDL without collapsing ARR too far.[47]

⁂

46. `[DESIGN] (Design)` How can persona skill profiles (analysisSkill, composure, worldviewRigidity) be mapped into metric modifiers so that different personas experience the same runContract as easier or harder without altering core invariants? [file:9]

47. `[DESIGN] (Design)` What range of UEC and EMD should be considered "comfortable mystery" versus "cognitive overload," and how should this influence default difficulty profiles for novice vs. expert runs in AI-Chat sessions?
48. `[DESIGN] (Design)` How can experience contracts ensure that every session includes at least one "cooldown window" where DET and CDL drop below certain thresholds, and how should the Director persona be required to acknowledge these moments in-world?
49. `[DESIGN] (Design)` What clear language can we give to designers to describe the difference between high SHCI but low AOS (deep, static haunt) and high AOS but

moderate SHCI (restless but shallow haunt), and how should each be used in session layouts?

50. `[DESIGN] (Design)` How should we define “liminal nodes” in node contracts (tagging with liminal:doorway, liminal:threshold, etc.), and what target LSG and HVF ranges should make a corridor feel unsettling versus neutral?

Here is a design-pass for 46–50 that keeps invariants fixed and uses persona/difficulty as metric-side modifiers only.[49]

46. Persona skills → metric modifiers

Persona skills should never change CIC/AOS/DET/SHCI envelopes; they only bias how **metrics** move inside a given runContract.[49]

A simple mapping:

- **analysisSkill**: scales how much UEC and EMD rise per clue and how much ARR increases when hints are noticed.[49]
  - High analysis: same run, but UEC and EMD climb faster on each clue, and ARR is easier to keep in the “satisfying ambiguity” band (fewer explicit hints needed).[49]
  - Low analysis: Director increases explicit hint density and caps per-event EMD deltas so the same run feels clearer and easier.[49]
- **composure**: sets how much DET and CDL each threat event contributes before auto-downshifts kick in.[49]
  - High composure: H.Director.constrainMetrics allows slightly larger DET deltas per beat inside the same runContract band; the persona needs fewer cooldown beats, so the run feels mechanically easier but emotionally “harder” (more sustained tension).[49]
  - Low composure: per-event DET deltas are clamped more aggressively, and the comfort policy forces more frequent cooldown windows, so the same layout feels safer.[49]
- **worldviewRigidity**: controls how many contradictions the persona will hold before collapsing uncertainty, i.e., it shapes CDL vs ARR.[49]
  - High rigidity: CDL rises quickly when clues conflict and ARR tends to drop toward “resolved” states, so the same run feels more punishing and less forgiving of ambiguity.[49]
  - Low rigidity: CDL is damped; ARR stays higher for longer, so the same run feels like “mystery you can live with” instead of cognitive overload.[49]

In practice, each personaContract can carry small per-metric weights like `UECWeight`, `EMDWeight`, `CDLWeight`, `ARRWeight` derived from these three skills. H.Director then applies these as multiplicative factors or caps when it calls `constrainMetrics`, making the same runContract feel easier or harder without moving the invariant bands.[49]

47. Comfortable mystery vs overload (UEC, EMD) and difficulty defaults

You already sketched difficulty bands per run (Novice/Standard/Expert) in terms of UEC, EMD, CDL, ARR. A usable split:[49]

- **Comfortable mystery**:
  - UEC roughly 0.4–0.7: players feel engaged and slightly unsure but not lost.[49]
  - EMD roughly 0.1–0.4: there are clues and open questions, but not so many that threads blur together.[49]
- **Cognitive overload**:
  - UEC above about 0.75–0.8 for extended stretches: constant uncertainty with no stable footing.[49]
  - EMD above about 0.5–0.6: too many active mysteries/clues at once.[49]
  - Typically coupled with high CDL, which is what makes overload feel unpleasant rather than intriguingly dense.[49]

Default difficulty profiles for AI-Chat sessions can then be:

- Novice: UEC 0.4–0.6, EMD 0.1–0.2, CDL capped around 0.5, ARR high.[49]
- Standard: UEC 0.5–0.7, EMD 0.2–0.4, CDL mid-range, ARR mid-to-high.[49]
- Expert: UEC 0.6–0.8, EMD 0.3–0.6, CDL allowed higher, ARR mid.[49]

Experience/runContracts should encode these bands per difficulty; H.Director and H.Selector then adjust clue density and branch complexity so novice runs stay in the comfortable mystery pocket while expert runs intentionally flirt with the overload boundary without breaking CDL caps.[49]

48. Guaranteed cooldown windows (DET/CDL) and in-world acknowledgement

Director comfort policies already include `maxDetPerSession`, `requiredCooldownRatio`, and `allowedCdlSpan`. To guarantee at least one cooldown window:[49]

- At the **contract/policy** level, enforce that every experience/runContract references a director comfort policy where `requiredCooldownRatio` is > 0 and that, for standard/mature tiers, there is an explicit "cooldown" stage or node class (safe room, hub after climax) whose target DET and CDL bands are low (for example DET in 0–2, CDL in 0.1–0.4).[49]
- At the **runtime** level, H.Director/H.Budget must track recent DET and CDL and refuse further high-DET patterns once the planned cooldown window is due, forcing selection of low-DET, low-CDL moods and events until metrics fall back inside the cooldown bands.[49]

To require in-world acknowledgement, director personas can:

- Mark cooldown beats with a narrative strategy verb like *cooldown* or *reassure*, and AI-Chat templates for these beats must include some diegetic acknowledgement: the

lights come up a little, the archivist speaks more plainly, a door that was locked now opens.[49]

- Telemetry can then correlate these flagged cooldown segments with DET/CDL drops to confirm policies are being honored.[49]

This keeps cooldowns from being invisible metric dips; they become part of the fiction while still being governed by numeric thresholds.[49]

49. Deep/static vs restless/shallow haunts (SHCI, AOS) and layout use

You already describe SHCI as spectral-history coupling and AOS as archival opacity/oscillation. Clear language for designers:[49]

- **High SHCI, low AOS – "deep, static haunt"**:
  - "This place is tightly welded to one or a few well-documented traumas; the story is clear, the ghost is local, and history is heavy but legible."
  - Numerically: SHCI band high (for example $0.7–1.0$), AOS band modest (for example $0.2–0.5$).[49]
  - Use for **anchor nodes**: hubs, sanctums, final rooms where the player gets stable information and where ARR tends to drop a little (some mysteries actually resolve).[49]
- **High AOS, moderate SHCI – "restless, shallow haunt"**:
  - "There are many conflicting fragments, rumors, and edits; something is wrong here, but the connection to any one event is thin and restless."
  - Numerically: AOS high (for example $0.6–1.0$), SHCI mid-range (for example $0.4–0.7$).[49]
  - Use for **liminal runs and side corridors** where you want high ARR and EMD but few hard answers: repetition, glitches, contradictions, and misdirections.[49]

Session layouts can then treat deep/static nodes as structural load-bearing rooms near the apex or resolution, and restless/shallow zones as the mist you move through: corridors, thresholds, optional loops where AOS spikes and SHCI is high enough to feel "real" but not enough to settle the legend.[49]

50. Liminal nodes, tags, and LSG/HVF targets

Node contracts already anticipate `liminalTags` (doorway, stair, threshold, etc.) with LSG and HVF used to drive liminal stress and haunt vectors.[50] [49]

Definition and bands:

- **Liminal node**: any node whose function is primarily to connect spaces or states and is tagged with one or more liminal tags, such as `liminal:doorway`, `liminal:stairUp`, `liminal:tunnelMouth`, `liminal:thresholdOther`. In contracts, this is expressed as `role` in a liminal class plus non-empty `liminalTags`.[50] [49]

- Unsettling corridor / threshold:
  - LSG band relatively high (for example $0.6–1.0$): crossing raises stress.[50] [49]
  - HVF.mag band moderate-to-high (for example $0.4–0.8$): haunt "pressure" is focused here.[50] [49]
  - These become preferred sites for transitions, glitch events, and spectral echoes in historySelector patterns.[50] [49]
- Neutral corridor:
  - LSG band low-to-mid (for example $0.2–0.5$), HVF.mag low-to-mid (for example $0.1–0.4$); either no `liminalTags` or only generic ones.[50] [49]
  - Used for navigation and pacing; selectors avoid stacking high-DET or high-CDL events here so they function as connective tissue.[50] [49]

Design guidance can then say: "Tag any doorway or transitional stair node as liminal, push its LSG above 0.6 and HVF.mag above 0.4 if you want it to feel wrong, and keep LSG below 0.5 and HVF.mag below 0.4 if you want it to feel neutral." Selectors and engine VFX/audio hooks read those bands to decide where to route haunt events and where to let players breathe.[50] [49]

⁂

51. `[DESIGN] (Design)` What small catalog of "Director strategies" (e.g., clinical recounting, ritual invocation, archival detachment) should we define, and how should each map to characteristic metric bands and tone labels? [file:9]

52. `[DESIGN] (Design)` How can we ensure that AI-Chat templates for different experience contracts never drift into content that violates explicitness ceilings and prohibited themes even when targeting high UEC and EMD, using only implication and tone?
53. `[DESIGN] (Design)` What does an "ideal" session curve look like for UEC, STCI, and DET over 20–40 turns, and how can we provide reference graphs designers can treat as targets when authoring new runContracts?
54. `[DESIGN] (Design)` How should we differentiate "branch density" (number of narrative forks) from "mystery density" (EMD) so that sessions can be highly replayable without becoming exhausting, and what ARR ranges make repeated play interesting rather than frustrating?
55. `[DESIGN] (Design)` What design patterns are best for signaling consent and safety changes in-world (e.g., lights changing, music shifts, meta-narration) so users feel informed without breaking immersion, and how should directors vary these patterns by persona type?

Here's a compact design pass for 51–55, wired to the existing director persona, template, and metric/invariant spine.[52] [53] [54]

51. Small catalog of Director strategies

You already have director personas (Cold Overseer, Ritual Statistician, Silent ArchivistDirector) with tone labels and metric envelopes; strategies should sit one level below as behavioral "styles" that shape how those bands are used.[53]

A useful minimal catalog:

- **Clinical recounting**
  - Tone: **clinical**, matter-of-fact, diagnostic.
  - Metrics: mid UEC $0.4–0.7$, low-mid EMD $0.1–0.3$, mid STCI $0.4–0.6$, mid-high CDL $0.5–0.7$ but with good structure, mid ARR $0.5–0.7$.[53]
  - Use for directors like Cold Overseer when you want the world to feel dissected and explained but still dissonant; the strategy prefers templates tagged "clinical explanation" and avoids highly emotive language even at higher DET.[53]
- **Ritual invocation**
  - Tone: **ritual**, incantatory, symbolic.
  - Metrics: higher UEC $0.6–0.8$, higher EMD $0.3–0.5$, moderate STCI $0.4–0.7$, higher CDL $0.6–0.8$, high ARR $0.7–0.9$.[53]
  - Maps well to Ritual Statistician personas, with templates tagged "ritual cadence" and "symbolic hint"; wording leans on repeated phrases and implied cause-effect rather than direct explanation.[53]
- **Archival detachment**
  - Tone: **archivist**, cataloging, almost bored.
  - Metrics: high UEC $0.6–0.9$, low-mid EMD $0.2–0.4$, higher STCI $0.5–0.8$ because of clear safe vs unsafe descriptions, mid CDL $0.4–0.6$, high ARR $0.7–0.8$.[53]
  - Fits Silent ArchivistDirector personas; templates emphasize dates, catalog codes, and redaction, often acknowledging AOS and SHCI in-world via missing pages, blurred recordings, etc.[53]
- **Gentle guide** (for lower tiers/novice design sessions)
  - Tone: **soft**, patient, teaching.
  - Metrics: UEC $0.4–0.6$, EMD $0.1–0.3$, STCI $0.3–0.5$, CDL $0.2–0.5$, ARR $0.7–0.9$.[53]
  - Uses templates tagged "explain concept", "reassure"; tends to lower CDL and keep DET modest, even when bound to darker regions.[53]
- **Ominous implication**
  - Tone: **whispered**, suggestive, off-screen.
  - Metrics: UEC $0.6–0.8$, EMD $0.3–0.5$, STCI $0.4–0.7$, CDL $0.5–0.7$, ARR $0.7–0.9$.[53]
  - Strategy favors "imply" in H.Director.applySafetyDecision and templates that talk around events, never describing impacts directly; ideal for higher tiers that still require implication-only output.[53]

Each strategy is just a small contract block (strategyId, toneLabel, metric bias/bands) that H.Director attaches to the persona envelope so templates can choose style and target ranges consistently.[52] [53]

52. Keeping templates within explicitness ceilings at high UEC/EMD

The existing safety stack already maps consent, tier, and explicitness into budget and boundary decisions that produce verbs like allow/soften/imply/refuse. To keep templates from drifting:[52] [53]

- **Contract-level constraints**
  - ai-chat-template contracts should declare explicitness modes and valid invariant/metric regimes, for example `explicitnessLevel = implicationOnly`, `validInvariants.LSG`, `validInvariants.SHCI`, and metric bands that never demand high DET/CDL in low-tier templates.[52] [53]
  - H.Templates.select must filter templates by both explicitness flags and the current aiChatHorrorProfile and director persona envelope before they are even considered.[53]
- **Runtime safety adapter**
  - Every candidate output goes through H.Safety and H.Director.applySafetyDecision, which maps boundary results into strategy verbs; templates for high-EMD, high-UEC beats should be written to support "imply" and "soften" variants as first-class branches rather than bolted-on edits.[52] [53]
  - ImageHelper and other hint systems are already required to output implication-only descriptions and opaque clue tags, with CDL caps enforced numerically per runContract and difficulty.[53]

By combining explicitness metadata on templates, strict selection filters keyed to aiChatHorrorProfile, and strategy verbs that force implication/softening when budgets or boundaries demand it, you can hit high UEC/EMD via density of symbols and contradictions, not via explicit depictions.[52] [53]

53. Ideal session curves for UEC, STCI, DET over 20–40 turns

Session route envelopes already log metric snapshots per step, which gives you the scaffolding for reference shapes. For a 20–40 turn horror session, an "ideal" reference for standard difficulty might look like:[54] [52]

- **UEC (uncertainty engagement)**
  - Start mid 0.4–0.5 as the scenario is introduced.
  - Rise to 0.6–0.7 through mid turns as mysteries and liminal nodes accumulate.
  - End slightly lower 0.5–0.6 if some threads resolve, or hold high 0.6–0.7 for more open endings.[52] [53]
- **STCI (safe-threat contrast)**

    - Start low-mid $0.3–0.5$.
    - Oscillate in a sawtooth pattern between safe and threat segments, with 2–3 noticeable peaks where STCI reaches $0.7–0.8$ for big events.[52] [53]
    - Final turns either stabilize at mid $0.4–0.6$ (cooldown/denouement) or spike one last time then drop.[52] [53]
- **DET (dread exposure)**
    - Begin near low $1–2$ (tutorial/safe intro).
    - Accumulate through 2–3 major peaks where DET rises sharply then drops during cooldown windows enforced by comfort policies and requiredCooldownRatio.[52] [53]
    - For standard tiers, keep cumulative DET area under curve within the configured band (for example $4–6$), with the last 3–5 turns trending downward to avoid runaway distress.[54] [53]

You can encode these as normalized reference curves in a small "metric-curve-preset" contract (for 20, 30, 40-turn runs), then have authoring tools overlay live or simulated trajectories from candidate runContracts on top of those graphs so designers can see where a run overshoots or undershoots target UEC/STCI/DET bands.[54] [52]

54. Branch density vs mystery density, and ARR for replayability

You already distinguish UEC/EMD (mystery) from structure (branches, nodes) in runContracts and node contracts.[54] [52]

- **Branch density**
    - Purely structural: number of forks per N turns or per critical path length; governed by graph topology (number of choices per node, depth of side paths).[54]
    - High branch density creates replayable permutations but can exhaust players if combined with high EMD and CDL, because each run must process too many distinct states.[52] [53]
- **Mystery density (EMD)**
    - How many **active** mysteries/clues exist per segment; encoded as EMD bands in run/node metricTargets and influenced by trigger and template design.[52]
    - You can have high branch density with low-moderate EMD (lots of paths, each relatively clean), or low branch density with high EMD (few paths but many layered mysteries).[52] [53]

For replayable but not exhausting sessions:

- Keep **branch density** moderate on any single run (for example, meaningful 2–3 way choices at a few key nodes) but ensure alternate branches gate different subsets of mysteries, so EMD per run stays within the comfortable range $0.2–0.4$ while global EMD across all runs is higher.[53]
- Target ARR bands around $0.6–0.85$ at end of each run: enough unresolved content to justify another play, but with some cores resolved so CDL is mid-range $0.4–0.6$.[52] [53]

Telemetry on UEC/EMD/ARR across multiple runs can then flag runContracts that combine high branch density with sustained high EMD and high CDL—those tend to feel exhausting rather than intriguing and should be tuned by reducing per-run mysteries or by rebalancing ARR so each ending closes at least one major thread.[54] [52]

55. In-world signaling of consent/safety changes, per persona

The "safety as in-world mechanic" work already calls for downshifts and refusals to appear as diegetic behaviors, not raw warnings.[53]

Design patterns:

- **Lights and color**
  - Cooldown or downshift: slightly brighter or warmer lighting, reduced flicker, fewer visual artifacts.
  - Approaching caps or boundaries: subtle desaturation, narrower light cones at liminal nodes, but never explicit gore or shock content beyond the tier.[52] [53]
- **Music and sound**
  - Safety raises: music simplifies, reverb tails shorten, noise layers drop out—mapping directly to lower DET and CDL bands via H.Audio RTPCs.[54] [53]
  - Intensity warnings: motif repetition, low drones, or "archival" sounds (tape hiss, projector click) can mark that the Director is at a limit without stating it outright.[53]
- **Meta-narration and Director voice**
  - Clinical personas (Cold Overseer): calmly state constraints in-world, for example "Further detail is not available in this record," when boundary modules force imply/refuse.[53]
  - Ritual personas: rephrase limits as ritual rules ("The words stop here; to continue would break the vow.").
  - Archivist personas: lean on redactions and missing pages ("This page is burned; the rest is filed under a sealed classification.").[53]

Implementation-wise, H.Director.applySafetyDecision already returns strategy verbs plus style hints; templates for each persona type should map "soften/imply/refuse/downshift" into one of these in-world signals instead of generic safety text. That keeps users informed about consent and intensity shifts while preserving immersion and tone.[52] [53]

⁂

56. `[DESIGN] (Design)` How can we encourage designers to think of node graphs as “dread fields” shaped by CIC, SHCI, and LSG, and what visualization tools (heatmaps, gradient fields) should we provide to make that intuition tangible?

57. `[DESIGN] (Design)` What naming conventions should we use for moodContracts and eventContracts so AI-Chat can infer their likely metric effects (e.g., `mood.liminal_hum.v1` suggests moderate UEC, low DET, medium LSG)?
58. `[DESIGN] (Design)` How should we describe “death” in story-driven runs so that it remains purely an implied branch (never explicit depiction), and how can metrics and director behavior make non-lethal but high-stakes outcomes feel just as consequential? [file:9]
59. `[DESIGN] (Design)` What principles should guide the design of image-helper prompts so that each generated image contains at least one interpretable clue for novice difficulty, while still allowing more abstract symbolism at higher difficulties without breaking CDL caps? [file:9]
60. `[DESIGN] (Design)` How can we construct a small library of “interpretation tools” (Pattern Recognition Lens, Doubt Amplifier, Ritual Decoder, etc.) and define their metric effects so designers can mix and match them to create distinct analytical personas? [file:9]

For 56–60 you can lean directly on the same spine you used in 51–55: tile-class bands, metricTargets, and H.Dungeon/H.Node as the “physics,” and templates/personas as the skin. [55] [56]

56. Node graphs as “dread fields” + visualization

You already treat each node/tile as a bundle of invariants; the easiest way to make designers think in “dread fields” is to treat the graph as a scalar field over CIC, SHCI, and LSG, with HVF as the implied flow. In practice, give them a view where every node in a dungeonRunContract or node-graph is colored by one chosen invariant (CIC, SHCI, or LSG), and edges are thin arrows that inherit the stronger of the two endpoint values to suggest “dread flowing downhill.” [57] [55]

On the tooling side, H.Dungeon.sampletile and H.Node.describe already give you a per-node snapshot of invariants and metrics, so the “dread field” visualizer is just a client that polls those APIs and renders a 2D projection: a heatmap background for CIC, a contour-like overlay for LSG across liminal tags, and optional vector glyphs for HVF direction if you introduce it. For designers, you can define a few presets in a runtime-dungeon-routing or debug-UI doc: “Liminal Ridge view” (emphasize high LSG, SHCI), “Catastrophe Basin” (CIC clusters), and “Comfort Shelves” (low DET, low LSG islands), so they start thinking in gradients and flows rather than isolated nodes.[56] [55] [57]

57. Naming conventions for moodContracts and eventContracts

You already have compact IDs like dungeon-run.mansion1flobbyDC.v1; the same pattern can encode expected metric effects in mood and event names so AI-Chat can infer intent without reading the whole card. A simple convention is:[55] [56]

```
`mood.<space>_<dominantCue>.<bandHints>.vX` where `<bandHints>` is a short code ove
```

For eventContracts, use an action-first stem and a compact delta code, e.g. `event.door_half_open.dUEC+10_dDET+15_dARR-05.v1` meaning this event intends to bump UEC and DET while slightly reducing ARR. The important part is that these hints are *redundant*: the real source of truth is still the metricTargets/invariantBands in the JSON, but having the codes in IDs lets H.Selector and H.Director do cheap, string-based filtering when building rough curves or explaining choices to designers in AI-Chat.[55] [56]

58. "Death" as implied branch + non-lethal stakes

Given your explicitness caps and content-boundary rules, you want "death" to be a routing outcome, not a described spectacle. In story-driven runs, you can standardize a pattern where physical death is always referred to indirectly as "the run ending in the dark," "your route closing," or "the house absorbing your attempt," and the engine implements this as a branch to a terminal node with high ARR and a short, non-graphic epilogue, never any depiction.[56] [55]

To keep non-lethal but high-stakes outcomes feeling consequential, you can use DET and ARR together: for "fail but live" branches you spike DET locally, then drop it sharply while *lowering ARR* (more closure), whereas for "implied death" you keep DET moderate but push ARR high and cut UEC, making the experience feel unresolved and uncanny instead of breathless. Director personas and comfort policies then treat "death" outcomes as hard session termination: H.Director.applySafetyDecision always chooses a downshift or imply strategy into a cooldown moodContract and instructs H.Run to end the run, so death is a quiet off-screen fade, not escalation.[55] [56]

59. Image-helper prompts, clues, and CDL

Your existing image-prompt research leans on structured descriptions per tile/room; you can make that difficulty-aware by tying "clue density" and abstraction to CDL and UEC bands per session or per node. For novice difficulty, define an image-helper policy that requires at least one explicit, interpretable clue object per prompt—e.g., a recognisable symbol, path, or landmark that directly maps to a puzzle or choice—and keep CDL low by avoiding layered symbolism that contradicts itself.[57] [56]

At higher difficulties, you allow more abstract symbolism by increasing EMD and ARR bounds while capping CDL via policy: images can contain multiple candidate clues or ambiguous motifs, but the prompt authoring guidelines require that at least one "anchor" element still cleanly maps to a real affordance in the node or event contract. You can express this as a tiny

schema or contract for image-helper configs per tier/difficulty that AI-Chat uses when writing prompts: min number of anchor objects, max allowed "pure mood" elements, and a target CDL delta (never more than a small increase per image) so visual ambiguity adds to mystery but doesn't swamp comprehension.[56]

60. "Interpretation tools" library and metric effects

You already talk about analytical personas and lenses; the cleanest way to operationalize them is as small, composable contracts—"interpretation tools"—that define how to transform metrics for the *user's sense-making layer* without altering the underlying world invariants. Each tool is effectively a set of local multipliers or offsets on UEC, EMD, CDL, and ARR as perceived by that persona, plus some template hints.[56]

For example, a **Pattern Recognition Lens** could slightly raise perceived EMD and lower CDL by summarising and connecting clues more clearly, making runs feel more solvable without changing actual branch density. A **Doubt Amplifier** could increase UEC and ARR while nudging CDL up in small steps, encouraging the director to surface contradictions and withhold resolution. A **Ritual Decoder** can reduce ARR by offering symbolic interpretations that tie disparate events together, trading some uncertainty for a sense of hidden structure, while keeping DET and STCI within the persona's comfort policy.[56]

You can model these as tiny contracts (e.g., `interpretationTool.pattern_recognition.v1`) that carry "metric perception deltas" and tags, and let director personas reference a set of tools in their config. At runtime, H.Director takes the base metric snapshot for the session and applies the active tools before choosing tone and templates, so mixing tools (Pattern Recognition Lens + Doubt Amplifier) yields distinct analytical personas without inventing entirely new director classes.[56]

⁂

61. `[DESIGN] (Design)` What guidelines should we give designers for using SHR (SHCI) and SPR together to define how "present" a spectral history is at a location, and how should this influence whether an encounter is described as a glitch, echo, or full apparition?

62. `[DESIGN] (Design)` How should we calibrate ARR ceilings for different experience types to ensure some runs always end with strong closure (low ARR) while others deliberately leave unresolved threads (high ARR), and how should UI communicate this to players?
63. `[DESIGN] (Design)` What policy should we adopt for limiting cumulative DET across a multi-session campaign, and how should directors and selectors coordinate to enforce campaign-level exposure caps while still allowing intense individual episodes?

64. `[DESIGN] (Design)` How can we define a small set of “safety personas” that AI-Chat can switch into when guardrails trigger, and what metric bands and tone descriptions should they follow so downshifts feel caring rather than punitive?
65. `[DESIGN] (Design)` How should we balance the desire for “metagame puzzle solving” (players analyzing invariants, sessions, and runs) against the risk of making horror feel purely mechanical, and what ranges of UEC and EMD best support that meta layer?

For 61–65 you can treat SHCI/SPR, ARR, DET, and UEC/EMD as “design dials” that are always interpreted through the same runContract, director, and safety layers you already have.[58] [59]

61. SHCI + SPR → glitch, echo, apparition

Designers should think of SHCI (spectral-history coupling) as “how tightly this node is bound to a specific past event” and SPR as “how likely that coupling is to support an actual manifestation right now.” A good guideline is to define three overlapping bands in the spine or a small “spectral-presence policy” doc:[59] [58]

- Glitch: low–mid SHCI (e.g. 0.2–0.5) and low SPR; presence is intermittent, non-responsive, easily dismissed as sensor noise or a memory misfire.
- Echo: mid–high SHCI (0.5–0.8) but moderate SPR; the past is strongly coupled, but manifestations are partial and mostly non-interactive—replays, shadows, repeating sounds.
- Apparition: high SHCI (≥0.7) and high SPR; both history and conditions support a fully present, responsive entity.

You can encode these ranges as policy bands that H.Node.describe or a small helper maps to a presenceCode glitch/echo/apparition, and then AI-Chat templates pick language accordingly while still keeping all horror implied. For example, a node with SHCI 0.75 and SPR 0.8 in a liminal stairwell would be tagged apparition, leading to richer off-screen effects (breath, voice, movement) than a low-SPR glitch, which might only get a single visual smear or audio dropout.[58] [59]

62. ARR ceilings per experience type + UI

ARR in your NODEGRID example is already a “how many mysteries stay unresolved” knob; you can standardize a few experience contracts with different ARR ceilings and surface them as presets. For instance:[59] [58]

- “Closed Case” runs: ARR target band 0.1–0.3, EMD moderate, CDL low—aiming for strong closure and minimal dangling threads.
- “Lingering Questions” runs: ARR 0.4–0.6, higher EMD, moderate CDL—some resolutions, some intentional gaps.
- “Unresolved Haunt” runs: ARR 0.7–1.0 (like the mansion lobby example), higher UEC and ARR with carefully bounded CDL so ambiguity feels intriguing rather than confusing.[58] [59]

UI can present these as simple sliders or labeled buttons ("Tidy ending", "Mixed", "Hauntingly unresolved") backed by the experienceContract that sets metricTargets. Designers then tune dungeonRunContract.metricTargets.ARR to sit entirely inside the chosen contract band, and a small HUD blurb or session setup card explains to players that "this run favors clear answers" or "this run will leave some mysteries deliberately open." [59] [58]

63. Campaign-level DET caps

Your director comfort policies already define maxDetPerSession and requiredCooldownRatio; campaigns just need a higher-level envelope that sums DET across runs and tells directors/selectors how aggressive they may be. A simple policy is:[60] [59]

- Define a campaignDetBudget (normalized or in "DET-units") and a per-session cap that must be a subset of both the consent profile and campaign budget.
- After each run, record the session's DET curve in the campaign state (via telemetry or a compact "campaign route" envelope) and decrement the budget accordingly.[59]

H.Director and H.Selector can then consult this campaign state before planning a new run: if remainingDetBudget is low, selectors prefer patterns and runContracts whose DET bands sit lower, and directors enforce stricter cooldown ratios or more frequent downshifts even if individual sessions are allowed to spike. This lets you have a few intense episodes early in a campaign, with later runs automatically shifting toward lower DET, higher ARR or EMD (more mystery, less raw stress) as the overall budget is consumed.[60] [59]

64. Safety personas for downshifts

The director-persona work already suggests comfort policies and strategy verbs (allow/soften/imply/refuse); "safety personas" are just a small family of persona cards tuned to low DET, low CDL, and caring tone bands. You might define three:[60] [59]

- **Gentle Archivist**: DET band low, CDL low, ARR moderate, UEC moderate—tone is calm, explanatory, focusing on organizing events and offering meaning.
- **Guardian Guide**: DET low–mid but with clear cooldowns, STCI moderate (strong safe–threat contrast), CDL low; tone is protective, offering choices that increase safety and agency.
- **Neutral Operator**: DET very low, CDL very low, ARR low; tone is technical and distancing, focusing on summarizing and pausing rather than continuing horror.

Each safety persona's directorPersonaContract would have invariantBands narrowed (e.g., lower SHCI/DET use) and a comfortPolicy that enforces more low-DET beats per high-DET beat, plus stricter CDL spans than normal directors. When guardrails or consent/budget rules fire, H.Director.applySafetyDecision can switch narration to one of these personas (or temporarily "overlay" its styleHint) so the downshift feels like someone taking care of the player in-world rather than the system punishing them for triggering a rule.[60] [59]

65. Metagame puzzle-solving vs mechanical feel

You already frame UEC (uncertainty engagement) and EMD (mystery density) as core entertainment metrics; metagame players enjoy reading invariants and telemetry, but too much density or CDL pushes them into pure spreadsheet mode. A good target for "healthy meta" sessions is:[59]

- UEC moderately high (e.g. 0.5–0.8) to keep players engaged in inference and route prediction.
- EMD in a mid band (0.4–0.7) so there are enough clues and contradictions to analyze, but not so many that every tile is a puzzle node.[59]

You can treat CDL as a guardrail for metagame play: meta-friendly runs should keep CDL moderate (not low—some dissonance makes analysis fun—but clearly below the levels you reserve for experimental or research tiers). Experience contracts aimed at meta players can then explicitly state "high UEC, mid-high EMD, bounded CDL" and surface this in UI as "good for theory-crafting" modes, while more atmospheric, story-first runs use lower EMD and slightly lower UEC so players don't feel obliged to treat the world as a logic puzzle.[59]

⁂

66. `[DESIGN] (Design)` What naming and tagging scheme should runContracts use to make it clear which invariants are central to their themes (e.g., DET-heavy prison, SHCI-heavy cemetery), and how should AI-Chat present those tags as user-facing descriptors?

67. `[DESIGN] (Design)` How should we define the difference between a "guided tour" AI-Chat mode and a "co-authored story" mode in terms of metric bands, branch counts, and director persona roles, and what constraints should each impose on runtime decisions?
68. `[DESIGN] (Design)` What simple, consistent way can we summarize an experience contract to players (e.g., intensity meter, ambiguity meter, mystery meter) using projections of UEC, DET, and ARR, and how can we keep those summaries honest as contracts evolve?
69. `[DESIGN] (Design)` How should we incorporate real-world folklore and historical events into seeds and regions without ever representing real victims or trauma directly, relying instead on synthetic invariants and fictionalized metadata to preserve ethical boundaries?
70. `[DESIGN] (Design)` What is the right balance of shared vs. per-user lore: how much should SHCI and CIC-based haunt patterns persist across players for a given region, and how much should selector and director behavior adapt to each player's prior routes and comfort outcomes?

66–70 slot cleanly into the runContract / experienceContract / persona patterns you already have; below I'll keep each answer design-facing but grounded in those invariants and metrics.

[61] [62]

66. Naming and tagging runContracts by invariants

RunContracts already carry invariantBands and metricTargets; you can standardize a simple naming/tagging scheme so authors and AI-Chat both see which invariants are "central." One pattern is to add a `themeTags` array and a constrained `primaryInvariants` block in run-contract and dungeon-run-contract schemas, using short, invariant-coded tags like `DET-heavy-prison`, `SHCI-drenched-cemetery`, `AOS-fog-archive`. A run would then declare something like `primaryInvariants: { DET: high, SHCI: medium, CIC: high }` and tags `["DET-heavy-prison","CIC-war-ruin"]`, with validation checking that these labels match the actual bands (e.g., "high DET" only if DET min/max exceed profile thresholds).[62] [61]

AI-Chat can expose these tags in user-facing copy as short descriptors: "Dread-heavy prison run (sustained DET, mid uncertainty)" or "Deep-history graveyard run (high SHCI, dense CIC, moderate dread)," while still driving selection numerically from the invariantBands. For UI search and filters, you can let players pick from a small categorical palette ("Dread-heavy", "History-saturated", "Liminal-focused"), which maps directly to those tags and underlying numeric ranges, keeping the naming system both legible and machine-enforced.[61] [62]

67. Guided tour vs co-authored story modes

You already sketched experienceContracts and runContracts; "guided tour" vs "co-authored story" can be defined as two presets over the same surface. Guided tour runs would target lower branch counts, tighter STCI (clearer safe-vs-threat contrast), and higher ARR floors (fewer unresolved threads), with director personas like Silent Archivist or Guardian Guide that prioritize explanation, low DET spikes, and constrained CDL. Co-authored story runs can widen branch counts and ARR bands, raise EMD and UEC targets, and use more improvisational director personas (e.g., Ritual Statistician) that accept higher CDL and allow the player's choices to meaningfully alter which contracts are activated.[62]

Runtime constraints follow from those envelopes: guided tour mode instructs H.Selector and H.Node to prefer paths with limited branching and to down-rank highly ambiguous patterns when CDL is already high, while co-authored mode explicitly allows deeper trees and will avoid auto-corralling the player back onto a single canonical route unless comfort or budget policies demand it. The key difference is that in guided mode, the director is allowed to "override" some user choices in favor of clarity and safety (within declared bands), whereas co-authored mode treats user intent as primary and makes selector/director logic respect branch consequences as long as they stay within tier and comfort caps.[63] [62]

68. Summarizing experience contracts with meters

Experience contracts already specify metricTargets for UEC, DET, ARR; you can project those into three player-facing meters: an "Intensity" meter (DET-driven), an "Ambiguity" meter (ARR plus CDL), and a "Mystery" meter (UEC plus EMD). For each contract, you can define a simple mapping like: intensity = normalized DET band center, ambiguity = f(ARR, CDL), mystery = f(UEC, EMD), all clamped to 0–1 and displayed as labeled bars or dial icons in UI. These projections should be computed mechanically from the contract's metricsEnvelope rather than hand-authored, and CI can enforce that the displayed labels ("low", "medium", "high") match the numeric ranges to keep them honest as authors tweak numbers.[62]

To keep summaries honest over time, telemetry from run-session metrics (actual DET peaks, average ARR/UEC) can be periodically compared against the contract's declared bands. If a contract routinely produces higher DET or ambiguity than advertised, your tooling can flag it for revision or automatically tighten its bands, ensuring that the player-visible meters remain a reliable prediction of how the run will feel.[63] [62]

69. Using real folklore/history without depicting real trauma

The region/seed structure and invariant spine are a good fit for "inspired by" without "recreating" real-world harm: you can let designers attach references to historical eras, locations, or folk motifs only in synthetic metadata (e.g., `influenceRefs` or `folkloreMotifIds`) while the actual events and victims in the regionContractCard and seedContractCard remain wholly fictional. CIC, SHCI, and AOS bands can be tuned to echo the *type* of catastrophe (war, plague, industrial accident) rather than any specific real event, and any names, dates, or personal details are replaced by abstract labels and invented archives. A simple policy is to require that all "inspired by" links pass through a separate research catalog in a governance repo, and never store literal real-world victim data or specific tragedies in runtime schemas or seeds.[62]

AI-Chat then only sees the synthetic invariants and motif tags; templates talk about "a collapsed mining town where records are missing" or "a graveyard built over older foundations," but never about real places, people, or events. This keeps spectral patterns grounded (CIC/SHCI still tie to a believable history field) while maintaining a strict ethical boundary: real trauma informs the *shape* of the haunt field, never its literal content.[62]

70. Shared vs per-user lore and SHCI/CIC persistence

Your telemetry and run-session metrics work already assumes cross-session aggregates; you can treat CIC/SHCI fields as mostly region-level invariants (shared across all players), with per-user overlays that adjust selector and director behavior based on prior routes and comfort outcomes. At the shared layer, a region's CIC and SHCI bands define the global "haunt skeleton": certain nodes, rituals, or motifs always exist there, so some haunt vectors and legends should recur across players to give the world continuity. On top of that, per-user lore state—what mysteries they've seen resolved, which endings they reached, their recorded comfort outcomes—can bias H.Selector and H.Director to favor deeper branches in

previously visited threads or to soften/delay certain patterns if prior runs overshot DET or CDL.[61] [63] [62]

A good balance is to treat structural things (which seeds exist, which core legends are possible, high-CIC sites) as shared, while the *resolution state* of those threads (which explanations the player has accepted, which apparitions they've "met") is per-user and lives in their session and account-level metrics. That way, SHCI-driven haunt patterns remain recognizable from player to player, but the director and selector can adapt pacing, ambiguity, and which branches to reopen based on each player's personal history and comfort telemetry, without ever rewriting the underlying region contracts.[63] [62]

⁂

71. `[RUNTIME] (Engine)` How should H.Selector and H.Node coordinate to ensure that a "liminal hooks" pattern never routes players through more than N consecutive high-LSG nodes without inserting a lower-stress node to respect DET and CDL limits?

72. `[RUNTIME] (Engine)` What runtime rules should govern when a spectral entity is allowed to "step up" from a subtle presence (e.g., audio glitch) to a more direct manifestation, based solely on historical invariants (CIC, SHCI, SPR) and not on arbitrary timing?
73. `[RUNTIME] (Engine)` How can director personas use `H.Director.constrainMetrics` each turn to slowly steer UEC and EMD toward target bands without causing abrupt jumps that feel artificial or manipulative to players?
74. `[RUNTIME] (Engine)` What algorithm should `H.Node.choose_next` use when multiple nodes are tied for top score—random selection, deterministic tie-breaking, or director persona preferences—and how can this be made tunable per experience contract?
75. `[RUNTIME] (Engine)` How should runtime logic decide when to terminate a run early (e.g., if comfortOutcome trends to `tooIntense`) while still offering a narratively satisfying wrap-up, and what signals should be emitted to AI-Chat to switch into epilogue mode?

71–75 all sit on top of the history-selector, node, director, and runContract surfaces you already have; below I'll frame them as concrete runtime rules and expectations.[64] [65] [66]

71. Limiting consecutive high-LSG nodes for "liminal hooks"

For a liminal-hooks historySelectorPattern like `hsp-dungeon-liminal-hooks.v1`, you can give `H.Node.choosenext` a small bit of run-local state: `liminal_streak` and `last_nodes[]`, maintained in H.Run or DungeonMemory per session, not in the node layer itself. Each time `H.Node.choosenext` is called under that pattern, it evaluates candidate neighbors as usual, but before accepting the top candidate it checks `(LSG >= high_LSG_threshold)` and updates or resets `liminal_streak`; if the streak has reached N, the function filters out any further high-LSG candidates and forces selection of the best node whose LSG is in a "cooldown band,"

e.g., below a configured threshold and consistent with DET and CDL cooldown targets from the dungeonRunContract and director comfort policy.[65] [66] [64]

Coordination with H.Selector is simple: the pattern's `entertainmentTargets` block already carries DET and CDL caps; the liminal-hooks pattern can declare a `maxConsecutiveLiminalNodes` field and a `cooldownRoleFilter` (e.g., prefer `hub` or `saferoom` nodes) that H.Node respects when the streak limit is hit. In practice, H.Selector chooses the pattern for the beat; H.Node then enforces both the N-node streak cap and the current DET/CDL budget state by down-ranking high-LSG candidates and boosting nodes whose local bands and metric deltas push DET/CDL back toward safe ranges, ensuring liminal corridors come in bursts rather than endless runs.[66] [64] [65]

72. Step-up rules for spectral entities from history invariants

Your spectral behavior should be a pure function of CIC, SHCI, SPR (and optionally AOS), plus recent metric and comfort state, never wall-clock time; that makes "step-up" moments feel like the history is asserting itself rather than the engine firing a timer. A simple rule hierarchy is: subtle presence (e.g., audio glitch) is allowed whenever local SPR and CIC exceed low thresholds, SHCI is above a minimum, and DET/CDL are below soft caps; stepping up to a stronger manifestation (visual glitch, partial apparition) requires SPR and SHCI to be inside a pattern's preferred bands and ARR/UEC still within contract targets, while full apparition events are only eligible when all three invariants are high and comfort policy has not reached max DET per burst for the current stage.[64] [65] [66]

Concretely, each spectral mechanicContract can declare invariant windows for each manifestation tier and expected metric deltas; at runtime, `H.Surprise` or a dedicated spectral module evaluates the current node's invariants snapshot (CIC, SHCI, SPR) against those windows and the run's metric state before choosing the highest tier that does not violate director comfort policy or dungeonRunContract bands. Because eligibility is recomputed on each candidate rather than driven by timers, the entity feels like it coalesces where history is densest and where the player's experience envelope still has "room" for a stronger echo, not just because X seconds elapsed.[66] [64]

73. Using H.Director.constrainMetrics to steer UEC/EMD smoothly

The director adapter spec already expects `H.Director.constrainMetrics(sessionId, intendedMetrics)` to clamp per-turn targets to persona and consent bands without inventing new values for nil metrics. To avoid artificial jumps, director personas should treat this call as a gentle steering mechanism: each turn they compute `intendedMetrics` as a small delta toward their runContract's target bands (e.g., raise UEC or EMD by at most δ per step if currently below band), pass that through `constrainMetrics`, and then feed the resulting effective metrics into selector/template choices, not into any direct content edits.[64] [66]

You can enforce smoothness by adding per-turn and per-window change caps into the director comfort policy (e.g., `maxDeltaUECPerTurn`, `maxDeltaEMDPerTurn`, `maxDeltaCDLWindow`), and having `H.Director.constrainMetrics` clamp requested moves so that even if persona

logic tries to jump UEC from 0.2 to 0.8 in one turn, the adapter reduces that to the allowed step. Because templates still derive their behavior from the effective metrics plus invariants, and because these metrics update gradually according to observed telemetry (H.Metrics) and comfort policy rather than per-turn whims, players experience a slow drift toward more mystery or clarity instead of sudden, obviously scripted swings.[66] [64]

74. Tie-breaking in H.Node.choosenext

Node scoring already comes from a pattern-defined kernel (liminal tags, mechanic hooks, historyCoupling to SHCI, etc.); when multiple candidates share the top score, you can treat tie-breaking as a tunable policy described in runContracts or experienceContracts. A good default for replayable horror is "deterministic but seed-driven": H.Node sorts candidates by score, then for equal scores uses a stable secondary key such as a per-run pseudo-random sequence seeded from `runId` and `currentNodeId`, so replays with the same seed behave identically but different runs explore different neighbors.[65] [64] [66]

To make this tunable, runContracts or experienceContracts can include a `nodeTieBreakStrategy` field with values like `deterministic` (always pick lexicographically smallest nodeId), `seededRandom` (use run seed), or `personaWeighted` (ask H.Director for a small preference vector over tags or roles). H.Node.choosenext then reads that strategy from the bound runContract and applies the corresponding algorithm, and selector telemetry logs tie-break decisions so designers can see whether particular strategies correlate with better UEC/ARR or comfort outcomes for a given experience type.[64] [66]

75. Early termination and epilogue mode

Comfort outcome and budget policies already define `tooIntense` and related states; a run can terminate early when either H.Budget or H.Director determines that continuing within the current contract would exceed max DET per session or produce CDL beyond allowed spans. Rather than hard-cutting, H.Run should treat this as a stage transition: when comfort trends to `tooIntense` (e.g., DET area-under-curve and cooldown ratios violate director comfort policy), H.Director flags `mustEndSoon`, narrows metric targets toward lower DET/CDL and higher ARR, and H.Selector is instructed to pick patterns tagged as `epilogue` or `cooldown` only, often bound to safe nodes or remote descriptions rather than in-place threats.[66] [64]

At the same time, H.Run emits an explicit mode change to AI-Chat and the engine—e.g., `runStage = "Epilogue"`, `terminationReason = "comfortTooIntense"`—so templates and VFX/audio know to shift tone to de-escalation: doors closing behind the player, the entity withdrawing, archivist-style narration of what was learned, etc. Once an epilogue beat (or small sequence) completes and DET/CDL are within gentle bands again, H.Run marks the run as complete, logs a comfortOutcome of `tooIntense` or `endedEarlyWithEpilogue` in run-session telemetry, and prevents further high-intensity patterns from being scheduled for that session, giving the player a narratively complete but safely curtailed experience.[64] [66]

⁂

76. `[RUNTIME] (Engine)` How can we design a simple "heartbeat" mechanism where each turn AI-Chat calls into `H.Metrics.snapshot` and `H.Director.applySafetyDecision` to adjust not just text but also audio, VFX, and pacing in-game?

77. `[RUNTIME] (Engine)` How should selectors and directors react if BCI-derived signals (in lab or research tiers) indicate that a player is more distressed than DET and CDL predicted—should they downshift more aggressively, or adjust metric targets for future runs?
78. `[RUNTIME] (Engine)` What runtime strategies can be used to prevent players from "camping" in safe nodes with very low DET and CDL to avoid progression, while still respecting their choice not to confront high-intensity content?
79. `[RUNTIME] (Engine)` How should AI-Chat communicate in-world that a downshift has occurred due to safety or budget constraints (e.g., "the house seems to hold its breath"), and how can this messaging vary across director personas without revealing implementation details?
80. `[RUNTIME] (Engine)` What runtime checks should precede any node transition to ensure that required Dead-Ledger proofs are still valid (e.g., entitlement tokens not expired) before activating certain high-intensity or research-tier content?

Below are runtime-facing answers for 76–80, wired to the existing H.Run / H.Metrics / H.Director / selector stack and the dungeon/run contracts you already have.[67] [68]

76. Heartbeat using H.Metrics.snapshot and H.Director.applySafetyDecision

You can treat each AI-Chat turn as a **heartbeat** that synchronizes text, audio, VFX, and pacing with the numeric spine.[69]

At the start of every turn, AI-Chat or the engine calls a small wrapper, for example `H.Heartbeat.step(sessionId, gdctx)`, that internally does:

- `local metrics = H.Metrics.snapshot(sessionId)` to get current UEC, EMD, STCI, CDL, ARR, plus any DET exposure proxies.[67]
- `local safetyDecision = H.Safety.evaluateTurn(sessionId, candidateOutput, context)` once a candidate text/audio beat is drafted.[67]
- `local result = H.Director.applySafetyDecision(sessionId, safetyDecision, H.Budget.snapshot(sessionId))` to get a *strategy verb* (`allow`, `soften`, `imply`, `refuse`, `downshift`) and an optional `styleHint`.[67]

From there, a thin glue layer maps strategy + metrics into engine parameters:

- Text: template choice and redaction level routed by `strategy` (e.g., `imply` selects implication-focused templates).[67]

- Audio: RTPCs like `horror_intensity`, `mystery_layer`, and `silence_ratio` derived from DET, UEC, and ARR; high DET + `downshift` raises silence and soft ambiences.[68]
- VFX: camera shake, vignette, color grading intensity as functions of DET, CDL, and HVF/LSG in the current node.[68]
- Pacing: inter-message delay and maximum reply length scaled from DET and CDL; `downshift` enforces longer pauses and shorter text until metrics re-enter comfort bands.[69]

The **important constraint** is that nothing in the engine makes up its own "intensity"; all four channels read strategy + metrics that came through `H.Metrics.snapshot` and `H.Director.applySafetyDecision` on every heartbeat.[67]

A small Lua surface might look like:

```
-- luahpc/heartbeat.lua
local Heartbeat = {}

function Heartbeat.step(sessionId, context, candidateOutput)
  local metrics = H.Metrics.snapshot(sessionId)
  local budget = H.Budget.snapshot(sessionId)
  local safetyDecision = H.Safety.evaluateTurn(sessionId, candidateOutput, context)
  local directorResult = H.Director.applySafetyDecision(sessionId, safetyDecision, b

  return {
    metrics = metrics,
    directorStrategy = directorResult.strategy,
    styleHint = directorResult.styleHint,
  }
end
```

Godot or CRust then bind audio/VFX parameters to `metrics` and `directorStrategy` rather than guessing.[68]

77. Reacting to BCI distress above DET/CDL predictions

For **research-tier** sessions, BCI / physiological signals should act as *hints* that modulate targets and strategies, not as a replacement for DET/CDL.[67]

At runtime, you treat BCI as an extra input into the per-turn pipeline:

1. The engine collects a distilled signal, for example `stressScore` in $[0, 1]$, or discrete bands `calm`, `elevated`, `high` from a BCI adapter.[68]
2. On heartbeat, you call something like `H.BCI.snapshot(sessionId)` to get the current band and confidence, and pass that into `H.Director.constrainMetrics` and/or a small `H.Director.applyPhysioHint` helper.[67]

The policy should be:

- **Within a run:** if BCI suggests distress higher than DET/CDL predicted, treat that as a reason to **downshift more aggressively**, not to chase higher DET.[67]

    - Narrow the effective DET band for this session (`H.Director.constrainMetrics` clamps intended DET downward).[67]
    - Bias director strategies toward `soften`, `imply`, or `downshift` until BCI returns to within expected bounds.[67]
- **Across future runs:** don't permanently change the content of the dungeon; instead, adjust **personalized metric targets** or aiChat horror profile bands for that user.[68]
    - A cross-session adaptation policy (already sketched for comfortOutcome `tooIntense`) lowers DET targets or raises ARR/CDL floors for subsequent runs with the same persona.[67]

In other words, BCI is an **adaptive safety overdrive**:

- It tightens DET/CDL envelopes on the fly and pushes director decisions toward safer strategies in this run.
- It feeds into run-session telemetry so that the user's future runContracts and personas start at gentler envelopes without modifying global dungeon invariants.[67]

78. Preventing "camping" in safe nodes while respecting avoidance

Camping happens when players stay indefinitely in low-DET, low-CDL nodes (e.g., `saferoom` tiles) to avoid progression.[67]
You want to discourage this *systemically* without forcing them into high intensity.

Use a mix of **soft pressure** and **bounded progression requirements**:

- **Soft pressure via metric drift:** define that staying many consecutive turns in a very low-DET node gradually **lowers UEC and EMD** (uncertainty and mystery), making the experience feel flat and less rewarding.[68]
    - `H.Metrics` can include an idle/camping detector: if `nodeRole == "saferoom"` and no progression input for N turns, apply small penalties to UEC/EMD per turn.[68]
    - Director personas then respond by gently nudging the player—e.g., more suggestive text ("the house grows still, as if waiting") but **not** raising DET.[69]
- **Non-coercive gating:** some core progression beats may require leaving the safe node, but you can ensure the first steps out are **low-to-moderate DET** rather than spikes.[68]
    - Selector patterns for exits from saferoom tileClasses can cap DET and CDL bands (narrower than global dungeon bands) so first steps feel tense but not overwhelming.[68]
    - Director comfort policies enforce required cooldown ratios, so after a high-DET segment you must route through these sanctuary segments anyway; staying longer just flattens UEC/EMD, not spiking intensity.[67]
- **Telemetry-based adjustment:** if many players camp and then quit, you can tune dungeonRunContract metricTargets so that extended safe-node time pushes ARR upward (more unresolved threads) without delivering closure, nudging design to offer **low-DET but high-EMD content** just outside the safe node.[68]

The core rule is: **never punish camping with surprise high-DET spikes inside safe nodes**; instead, let the experience become dull, gently encourage motion with low-intensity hooks, and reserve hard gates for actual objectives.[68]

79. In-world communication of safety/budget downshifts

When `H.Director.applySafetyDecision` yields `soften`, `imply`, or `downshift`, AI-Chat should **acknowledge the shift diegetically** without exposing the machinery.[67]

A good pattern is to attach a **styleHint** to the strategy (already suggested in the director API), for example `ritual`, `clinical`, `archivist`, then let persona tone decide wording.[67]

Concrete behavior:

- **Narrative framing**:
  - `downshift` in a haunted house persona: "The house seems to hold its breath; whatever was about to step through chooses to wait."
  - `imply` in an archivist persona: "The record grows vague at this point; the ink fades, and only hints remain of what was done here."
- **Consistency with strategy**:
  - `soften` should reduce explicitness while preserving event occurrence ("You hear something break upstairs, but the details blur the second you try to focus.").
  - `imply` moves the event offscreen or into aftermath clues; DET delta shrinks while ARR and EMD can still rise.[67]

To keep this **persona-flexible**, templates for "downshifted beats" should be tagged with:

- `safetyStrategyTag` (e.g., `safety-downshift`, `safety-imply`).
- `toneLabel` or director kind (cold overseer vs. ritual statistician) so the same safety outcome generates different phrasings from different personas.[67]

Internally, AI-Chat only sees `strategy` + optional `styleHint`, chooses templates tagged accordingly, and never branches directly on raw safety violations or budgets.[67]
Players thus feel an **organic lull or retreat** ("the thing withdraws") rather than "we're lowering intensity for safety reasons."

80. Dead-Ledger proof checks before node transitions

For **high-intensity or research-tier** content, the runContract and dungeonRunContract already carry optional `deadLedgerRef` fields that point to entitlement / attestation bundles.[68] [67]

Before transitioning into a node or pattern that relies on those entitlements, runtime must perform a **proof validity check**.

The pre-transition algorithm should be:

1. When `H.Node.choosenext` (or equivalent) proposes `nextNodeId` or a high-tier pattern, the engine constructs a small context:

```
local entContext = {
  sessionId = sessionId,
  runId = runContract.id,
  regionId = regionId,
  dungeonId = dungeonId,
  nodeId = nextNodeId,
  deadLedgerRef = nodeCard.deadLedgerRef or runContract.deadLedgerRef
}
```

2. Call a narrow validator, e.g. `H.Entitlement.validate(entContext)` that:
   - Verifies the referenced Dead-Ledger proof exists, is correctly signed, and not expired.
   - Confirms the player/account holds any required tokens or capability flags (tier, research consent, BCI agreement).[67]
3. If validation **fails**:
   - Mark the node or pattern as ineligible for this session (`entitlementStatus = "invalid"`).
   - Instruct H.Selector/H.Node to pick an alternate node or pattern at the same structural location but with lower tier / public content bands.[68]
   - Emit a structured signal to AI-Chat like `mode = "fallback-entitlement"`, so it can narrate a diegetic alternative ("the forbidden wing is locked; another corridor opens instead") rather than breaking immersion.[67]
4. If validation **passes**:
   - Proceed with the transition, logging the proof ID and tier in telemetry (e.g., in the history-selector-decision-event and any run-session metrics rows).[68]

This can be wired as a **mandatory gate** between node-choice and activation:

```
local choice = H.Node.choosenext(sessionId, regionId, currentNodeId, patternId)
local entResult = H.Entitlement.validate{
  sessionId = sessionId,
  runId = runId,
  regionId = regionId,
  dungeonId = dungeonId,
  nodeId = choice.data.nextNodeId,
  deadLedgerRef = nodeCard.deadLedgerRef or runContract.deadLedgerRef
}

if not entResult.ok then
  -- choose safer alternative and inform AI-Chat to shift into fallback / epilogue to
end
```

For **research-tier** nodes the same gate also checks that the session's tier and consent state match the proofs, preventing accidental leakage of experimental behaviors into public experiences.[67]

⁂

1. `[RUNTIME] (Engine)` How should selectors use encounter history (e.g., nodes visited, patterns resolved) to avoid repetitive beats in a single session, and what thresholds constitute "too much repetition" for different experience types?

82. `[RUNTIME] (Engine)` How can node scoring incorporate both local invariants and session-level metrics, e.g., awarding bonus weight to nodes that bring UEC or CDL closer to target bands, without producing obviously "gamey" routing behavior?
83. `[RUNTIME] (Engine)` What runtime rule should govern when to introduce puzzles or clue-based imagery (via image helper) versus pure atmospheric beats, given current EMD, UEC, and CDL, so puzzle sequences do not overwhelm the horror pacing?
84. `[RUNTIME] (Engine)` How should we decide when to introduce "false safe" moments (nodes that look low-DET but carry subtle spectral implications) while still ensuring that overall session DET remains within comfort policy bounds?
85. `[RUNTIME] (Engine)` How can AI-Chat and engine logic coordinate so that user choices at narrative forks directly influence metric evolution (e.g., brave choices increasing DET and UEC, cautious choices lowering DET but raising ARR) in predictable, tunable ways?

Selectors should treat encounter history and session metrics as first-class inputs to scoring, but always within the envelopes declared by the runContract, directorPersona, and comfort policy.[70] [71]

81 – Avoiding repetition with encounter history

Selectors should maintain a **session-level encounter history** keyed at least by `(regionId, nodeId, patternId, mechanicId, motifId)` with counts and timestamps for: "visits", "beats fired" (e.g., jumpscare, lore-beat, puzzle), and "patterns resolved". This history is already implied by the session route envelope and history-selector telemetry; you're just making it a runtime query surface for node selection.[71] [70]

For each candidate node or pattern, scoring should:

- Apply a **recency penalty** based on last-visit turn or time, using exponential decay over a fixed horizon (e.g., last N beats) so "freshness" recovers over time.[71]
- Apply a **motif repetition penalty** when the same pattern family or mechanic kind (e.g., `sanityTick`, `lightDrop`) has fired more than K times within a sliding window.[71]
- Use **experience-type presets** in the runContract or experienceContract to define repetition caps per family, for example:[70]
    - Slowburn Dread: allow more motif reuse with softer penalties (e.g., 3–5 similar beats per 30 turns) to keep a rhythmic, oppressive feel.

- Short Shock: cap sharp motifs more tightly (e.g., 1–2 high-DET spikes of the same kind in a 10–15 turn window) and prefer diversity in between.

A simple policy is: once a motif's "recent count" exceeds its configured window cap, you treat any node whose primary mechanic/tag is that motif as "cooldown-only": either excluded outright or only allowed when no safe alternative exists. For AI-Chat and design tooling, this stays transparent if you surface the reasons field from `H.Node.scorenextfromhistory` ("motif repetition penalty applied") so designers can see why certain nodes no longer come up.[71]

82 – Node scoring with invariants and session metrics (without feeling "gamey")

You already have the pattern: raw history/invariant features are scored, then fed through selector rules such as `SEL-R005 budgetfit` and historyCoupling. To incorporate UEC/CDL/ARR steering:[71]

1. Treat **local invariants** as the base fit: CIC, AOS, LSG, SHCI, DET, HVFmag, etc., via the same alignment logic used in node and tile classification.[71]
2. Treat **session metrics** as a small, bounded correction: how far current UEC, CDL, ARR are from the run's metricTargets band in the metricsEnvelope.[70]

A robust pattern is:

- Compute a **base score** from invariants, historyCoupling, liminal/mechanic hooks, and repetition penalties, as already sketched in `H.Node.scorenextfromhistory`.[71]
- Compute a **metric steering term** per candidate:
  - Predict the node's expected delta on UEC, CDL, ARR from its metricTargets and mechanicHooks (e.g., puzzles raise EMD and CDL, cooldowns reduce DET and CDL).[70]
  - Score how much that delta would move the current session snapshot toward the target band center for each metric, clamped to a small range (e.g., $[-0.5, +0.5]$).[71]
- Combine as a **weighted average**, not an unbounded sum, and normalize across candidates (min–max or softmax) so you only use relative ranking.[71]

To avoid "obvious gamey routing", keep these safeguards:

- Keep metric steering **low-amplitude**: invariants and historyCoupling dominate. Metrics act as a tiebreaker + slight preference for nodes that "heal" drifted metrics.[71]
- Never route purely by metric distance; always require the candidate to satisfy invariantBands and pattern-level constraints first.[70]
- Audit via telemetry: compare runs with and without metric steering to confirm that path variety remains high and players don't easily infer "the system is pushing me back to the center".[70]

This keeps node selection feeling like "the haunted place has moods" rather than "the game is rubber-banding my stats."

83 – When to introduce puzzles/clue imagery vs pure atmosphere

You already treat **EMD, UEC, CDL, ARR** as the experience spine; puzzles and image-based clues should be modeled as mechanics that tend to:

- Raise **EMD** and often **CDL**, sometimes also UEC.[70]
- Lower ARR if they resolve mysteries too cleanly, or raise ARR if they add new, contradictory evidence.[71]

A good runtime rule is:

- Only introduce a **puzzle or clue-heavy node** (including image-helper beats) when:
  - UEC is at or above the lower bound of its target band (player engaged enough to care).[70]
  - EMD is at or below the upper bound of its band (room for more structured mystery).[70]
  - CDL is below a configurable ceiling for the experience type (e.g., CDL < mid-high of band for Slowburn; even lower for novice modes).[71]

If any of these preconditions fail, you bias toward **pure atmospheric or clarifying beats** instead:

- If CDL too high: pick a node whose expected effect is to reduce CDL and DET (safe corridor, recap beat, simple image with obvious symbolism).[70]
- If EMD already high: favor atmospheric nodes with low EMD contribution (creaks, distant footsteps, visual oddities) so the session has time to metabolize existing mysteries.[70]

To keep pacing coherent, encode maximum **puzzle density** in the runContract or historySelector pattern (e.g., max puzzles per 10 turns, or per stage of the run) and let selectors treat puzzle mechanics like SurpriseTriggers with caps. Puzzles then become "expensive" nodes the director spends carefully, rather than a default.[71]

84 – Scheduling "false safe" moments without breaking comfort bounds

"False safe" nodes are structurally low-DET, low-CDL spaces that carry subtle spectral signals via SHCI, LSG, and HVF rather than explicit shocks. To govern them, treat them as a separate **beat class** in the director comfort policy and runContract:[70]

- DET envelope: the node's invariantBands.DET must be within the current comfort policy's "cooldown" or low band, even if its SHCI and LSG are high.[70]
- CDL: keep within or below mid-band; the point is relaxed tension with a hint of wrongness, not cognitive overload.[71]

Runtime policy:

- Insert a false-safe node when:
  - Cumulative session DET is rising toward the comfort policy's maxDetPerSession or toward a soft cap for the current stage.[70]

- requiredCooldownRatio is due: you owe some low-DET minutes/turns relative to recent high-DET beats.[70]

To keep them **false**, not genuinely safe:

- Ensure these nodes sit on **high-SHCI, high-LSG, or high-HVFmag** tiles, so subtle manifestations are justified by the history layer (e.g., a safe room whose window reflection is one frame out of sync).[71]
- Use selector patterns that prefer nodes with liminalTags or historyCoupling, but with low DET metricTargets, so the haunt is implied in the architecture, not in explicit events.[71]

DET remains within comfort bounds because the node's visible mechanics are low-intensity; SHCI and LSG drive **UEC/ARR** by making the safe space feel fragile and temporally unstable. Director comfort policies should explicitly count these beats toward cooldown quotas, so the system doesn't "cheat" and treat them as free high-intensity exposure.[70]

85 – Coordinating AI-Chat choices and engine metrics

The stack you already have (runContract + directorPersona + session route envelope) is the right place to bind **choice semantics** to metric deltas. The pattern is:[70]

1. Annotate **narrative forks** (mechanicContracts, node decisions, or AI-Chat templates) with:
   - A small, declarative **outcome vector** over UEC, DET, CDL, ARR (and possibly EMD, STCI).[71]
   - Semantic tags like `brave`, `cautious`, `avoidant`, `curious`, which director personas interpret into metric deltas via their comfort policy.[70]
2. At runtime, when AI-Chat presents options, it:
   - Pulls the fork's contract to know which choices are "brave" vs "cautious" for that run/persona.[70]
   - Maps user selection to a set of **intendedMetrics** deltas and passes them to `H.Director.constrainMetrics`, which clamps them to persona and consent bands.[71]
   - `H.Metrics` then records the updated snapshot in the session route envelope step, so selector and budget modules see the new state on the next beat.[70]

Typical mapping:

- **Brave choices**: increase DET and UEC within band, possibly raise EMD, while leaving ARR high by introducing new ambiguity instead of closing threads.[70]
- **Cautious choices**: lower DET, often lower CDL, but slightly **raise ARR** by leaving mysteries unresolved or by taking longer, safer routes that delay resolution.[71]

Tunability comes from:

- Keeping all deltas small and data-driven in contracts (e.g., per-choice `deltaDET`, `deltaARR` fields), then letting directorPersona + comfortPolicy clamp and bias them.[70]

- Logging per-choice deltas into telemetry (run-session metrics), so you can tune tables later if “brave” feels too punishing or “cautious” feels too flat.[70]

Coordination between engine and AI-Chat is then mechanical: engine exposes metrics snapshots and fork contracts via `H.Run`/`H.Node`, AI-Chat describes them to the user in narrative terms, and both sides treat the numeric deltas as authoritative.[71] [70]

⁂

86. `[RUNTIME] (Engine)` What runtime policies should be in place to prevent a director persona from aggressively pushing ARR (ambiguity) at the expense of closure when telemetry shows that many players rate those runs as unsatisfying?

87. `[RUNTIME] (Engine)` How should BCI or physiological data, when available, be incorporated as optional “hint signals” into metric adjustments (e.g., nudging DET or CDL bands) without ever becoming a direct or sole driver of horror intensity?
88. `[RUNTIME] (Engine)` What runtime strategy should be used to align template selection with selector and director decisions, so that narrative outputs, audio mood, and node transitions all tell a consistent story of rising or falling tension?
89. `[RUNTIME] (Engine)` How should the system handle “failed” puzzles or branches (user misses key clues repeatedly) in a way that preserves horror tone—e.g., by folding failure into the narrative rather than dumping the player out-of-world?
90. `[RUNTIME] (Engine)` What runtime data should be logged for each turn (nodeId, patternId, directorStrategy, metric snapshot, comfortPolicy status) to enable detailed reconstruction of sessions for debugging and research?

Director personas, BCI hints, templates, and telemetry should all be wired as thin layers on top of the same contracts and metrics envelopes, so they can never silently drift away from comfort policies or safety invariants.[73] [74]

86 – Policies to prevent over-pushing ARR (ambiguity)

You already treat ARR as a spine metric in directorPersona and runContract envelopes, and comfort outcomes like tooIntense / tooFlat are derived from route telemetry. To stop a director persona from abusing high ARR, you want three levels of guardrails.[75] [74]

First, constrain **ARR bands per experience type** at schema level: experienceContracts and directorPersonaContracts for “closure-heavy” modes simply cannot declare an ARR.max above a product-defined ceiling, and CI rejects any persona whose ARR band widens beyond its aiChatHorrorProfile and runContract caps. That prevents an Archivist-style persona from advertising “grounded investigation” while actually targeting full 0.9–1.0 ambiguity.[73] [75]

Second, add a **satisfaction-aware policy** in H.Director and H.Metrics: run-session telemetry already logs per-run UEC, CDL, ARR and a comfortOutcome. You extend this with a simple quality flag (e.g., `userSatisfactionScore` from in-product ratings, or proxies like frequent abandonment) and maintain a persona- and contract-scoped aggregate: if a director consistently produces runs with ARR high but satisfaction below a threshold, CHATDIRECTOR down-routes that persona automatically by:[75]

- Narrowing effective ARR bands for new runs of that experience type (clamping ARR.max toward a lower ceiling), and
- Applying an extra **resolution floor** near the end of the run: e.g., for the final N steps, require that ARR moves toward the lower half of the band unless the experienceContract explicitly declares "high-ambiguity ending".[74] [75]

Third, enforce **per-run ARR ceilings and slope rules** in H.Director.constrainMetrics: when a persona requests ARR deltas at each turn, the adapter clamps both absolute ARR and its rate of increase, and can require that late-stage turns cannot increase ARR at all for closure-oriented contracts. The director's strategy verbs (allow / soften / downshift / refuse) then include "closure bias" hints, forcing template selection and node routing to favor resolution beats over new mysteries once ARR is above the experience's comfort band or once telemetry has recorded multiple "unsatisfying" runs for this persona/contract pair.[74] [73] [75]

87 – Using BCI/physiology as "hint signals" only

The safety spine already assumes metrics like DET and CDL are derived from observable behavior and consent, not opaque signals. BCI or physiology for research tiers should be treated as **secondary evidence streams** that bias metric adjustments but never replace the contract envelopes or SAF invariants.[75]

Implementation-wise:

- BCI inputs (e.g., elevated heart rate, EDA spikes) are ingested into a small, tier-gated module (e.g., `H.BCI`) that outputs **confidence-scored hints**, such as "probable arousal up" or "probable disengagement".[75]
- H.Metrics then uses these hints as low-weight modifiers on its next DET / CDL estimates (or on the intendedMetrics fed into H.Director.constrainMetrics), with strict caps per turn (e.g., ± 0.1 within the band) and per window, so the BCI cannot cause large jumps.[74] [75]

Crucially, the **physics layer remains primary**:

- SAF invariants, consent caps, director comfort policies, and runContract metricTargets still define the allowed ranges for DET, CDL, and ARR; BCI hints can tilt which side of the band you sit on, but cannot widen bands or bypass downshift rules.[75]
- BCI features are only available in research-tier runs whose contracts explicitly declare their use in the metricsEnvelope; standard tiers ignore them completely, to avoid hidden adaptation.[75]

Telemetry then logs both metrics and the BCI hint stream so governance can verify that BCI-driven nudges correlate with improved comfort or engagement and do not

systematically push sessions into higher DET or CDL than declared.[75]

88 – Aligning templates, selector, and director for coherent tension

Your H.Run state machine already routes every turn through H.Selector, H.Metrics, H.Budget, H.Safety, and then template selection, with the director persona constraining effective metrics. The runtime strategy is to make **effectiveMetrics** the single shared "weather report" for tension and ensure every layer consults it before acting.[75]

At each turn:

- H.Director.constrainMetrics takes the run's intended metric movement (from experienceContract curves and stage, e.g., Outer → Locus → Rupture → Fallout) and clamps UEC, DET, CDL, ARR into persona + consent bands, producing `effectiveMetrics`.[73] [75]
- H.Selector.choosePattern is given both invariants and `effectiveMetrics`, and its rules select patterns that naturally drive the curve in that direction (escalation arcs, cooldown fillers, lore threads), subject to budget and SAF.[75]
- H.Templates.select receives `selectorPatternId`, node context, and `effectiveMetrics`, and only chooses templates whose own metric and invariant envelopes overlap those effective bands (using the planned ai-chat-template-contract validInvariants + metricTargets blocks).[75]

Audio and node transitions read from the same signals:

- H.Audio maps CIC, DET, LSG, and stage to RTPCs like `horror_pressure` and `liminal_intensity`, keeping ambience and stingers synchronized with the narrative's rising or falling tension.[73] [75]
- H.Node.choosenext uses pattern- and history-based scoring plus the same `effectiveMetrics` to favor nodes whose expected deltas match the curve (e.g., high-LSG corridors as you approach Rupture, low-DET safer rooms during Fallout).[74] [75]

Because all four stacks (templates, selector, audio, node routing) depend on the same numeric envelope rather than separate "moods", the session feels like one coherent story of tension rather than uncorrelated subsystems. Telemetry via run-session-metrics and selector-decision events then lets you verify that patterns, templates, and audio actually moved UEC, DET, and STCI along the intended curve.[74] [75]

89 – Handling failed puzzles and missed clues in-world

Your image-helper and puzzle designs already encode difficulty, metric deltas, and clue visibility in contracts (e.g., hiddenTags and per-difficulty CDL / UEC contributions). Runtime handling of repeated failures should follow the same doctrine: treat failure as **content**, not as an error, and keep it bounded by comfort policies.[75]

Mechanically:

- Puzzles and clue sequences are modeled as mechanicContracts with `expectedMetricDelta` on UEC, EMD, CDL, ARR and optional failure branches whose own deltas are gentler (e.g., less DET spike, more CDL reduction).[74] [75]
- Each mechanic tracks attempts and outcome in telemetry ("seen clue but no progress", "wrong answer", "gave up"), with caps per run in a pattern's entertainmentTargets (e.g., `maxPuzzleRetries` or `maxHighCDLFailuresPerStage`).[74]

At runtime, when attempts exceed thresholds or CDL drifts above the puzzle's band:

- H.Director.applySafetyDecision chooses soften or downshift for the mechanic's next beat, causing template selection to pivot to "diegetic hint" or "failure folded into myth" rather than a hard wall.[73] [75]
- H.Templates and node logic then narrativize the failure: doors that "refuse you and stay locked", rituals that "complete in the wrong way", or an Overseer persona that quietly reroutes the player down a different, less demanding branch while acknowledging what was lost.[74] [75]

Crucially, the system never dumps the player to an out-of-world screen; even when a branch is abandoned, H.Run terminates that mechanic by selecting low-DET, lower-CDL beats that make the failure feel like part of the horror, not a UI problem. Route envelopes log these pivots so designers can see where players struggled and adjust puzzle EMD/CDL bands or hint policies in future versions.[74] [75]

90 – Per-turn data for detailed reconstruction

The session route envelope spec already outlines the core identity and metric fields; for rich debugging of runtime behavior, you want to standardize a **per-turn record** that combines node, selector, director, safety, and budget context.[74] [75]

At minimum, each turn's telemetry event should include:

- Identity and contracts: `sessionId`, `regionId`, `runContractId`, `dungeonRunId` (if applicable), `nodeId` (current and next), `selectorPatternId`, `eventContractId`, `moodContractId`, `aiChatHorrorProfileId`, `directorPersonaId`.[74] [75]
- Director and safety: `directorStrategy` (allow / soften / imply / downshift / refuse from H.Director.applySafetyDecision), `comfortPolicyId`, a small comfort status (`detExposure` vs `maxDetPerSession`, `cooldownRatioSoFar`, whether within allowedCdlSpan).[73] [75]
- Metrics and budget: pre- and post-step snapshots of UEC, EMD, STCI, CDL, ARR, plus derived DET and any BCI hint flags, along with budget state (`perModality` usage, `overBudget` flags).[75] [74]
- Selector internals: a summarized `selectorDecision` blob with pattern, rule IDs applied (e.g., SEL-R004 escalationcurve), and top candidates with scores and reasons (liminal tags, mechanic hooks, SHCI fit).[74] [75]

Because the format is NDJSON-friendly and contains no raw narrative, analysts can reconstruct full metric trajectories, see which patterns and director strategies preceded comfortOutcome tooIntense or tooFlat, and pinpoint mis-tuned contracts or personas. This

same envelope is what future tuning tools and AI co-designers will read to adjust rule weights, ARR ceilings, puzzle difficulty, and director comfort policies without guessing.[75] [74]

⁂

91. `[RUNTIME] (Engine)` How can we design a “micro-run” concept (short, 5–10 turn experiences) for AI-Chat that still respects all invariants and metrics but is tuned for casual interaction, and what runtime differences should they have compared to full runs?

92. `[RUNTIME] (Engine)` How should selectors and directors behave when a session is resumed after a long gap—should they reintroduce context gently, re-establish DET and CDL from lower baselines, or pick up where metrics left off?
93. `[RUNTIME] (Engine)` What runtime policy should govern cross-session adaptation—for example, lowering DET target bands in subsequent runs if prior sessions repeatedly ended with comfortOutcome `tooIntense`, even when consent tier remains unchanged?
94. `[RUNTIME] (Engine)` How can director personas use `requiredCooldownRatio` to decide when to insert explicitly low-DET “sanctuary” segments, and how can those segments remain thematically aligned with the rest of the horror experience?
95. `[RUNTIME] (Engine)` What runtime behaviors should be reserved exclusively for research-tier sessions (e.g., certain mapping kernels, BCI-driven modulation) to prevent them from accidentally leaking into public-tier experiences?

Micro-runs should be first-class runContracts with shorter curves and stricter comfort rules, while resume, adaptation, cooldown, and research-only behavior all continue to flow through the same H. APIs and metric envelopes you already defined.[76] [77] [78]

91 – Designing micro-runs (5–10 turns)

Micro-runs can be modeled as a specific runContract family with explicit constraints: single region, single stage arc, and a fixed turn budget (e.g., 5–10 steps) declared in the contract. Their `metricsEnvelope` would define compressed UEC/STCI/DET curves (e.g., fast rise and quick cooldown) and tighter DET caps than full runs, while still obeying tier and consent ceilings from the spine.[77] [78]

Runtime differences versus full runs mainly live in H.Run and H.Selector: H.Run.currentStage advances faster (Outer → Rupture → Fallout in a handful of turns), and H.Budget / H.Director enforce lower `maxDetPerSession` and more aggressive cooldown insertion to fit the short duration. Micro-runs also prefer patterns marked for short formats (e.g., single-node encounters, minimal branch depth) and always end with an epilogue step that de-escalates DET and CDL into comfort bands before termination, so casual users can drop in and out safely.[78] [76] [77]

92 – Resuming sessions after a long gap

The session route envelope already records runContractId, metrics curves, and comfortOutcome per session, so you can distinguish "short break" from "long gap" at runtime. When a session resumes after a long gap, selectors and directors should treat it as a new **on-ramp** into the same run rather than blindly continuing high DET.[77] [78]

Policy-wise, CHATDIRECTOR can mark the resumed session with a "reentry mode" flag: H.Director.constrainMetrics then aims to re-establish DET and CDL at lower baselines for the first few turns (e.g., Outer / Threshold style beats), while H.Selector prefers recap patterns and safe nodes until metrics show stable engagement. After this reintroduction window, the director can gradually steer metrics back toward the original targets, using history (nodes visited, mysteries unresolved) from DungeonMemory / route envelopes to rethread context without replaying the entire run.[76] [78] [77]

93 – Cross-session adaptation policy

You already sketched persona- and profile-scoped telemetry aggregates that track DET exposure and comfortOutcome per run; cross-session adaptation is a straight extension. A small "comfort index" keyed by accountId, aiChatProfileId, and directorPersonaId can store rolling statistics such as `detAreaUnderCurve`, `detPeak`, observed cooldown ratios, and distribution of comfortOutcome (`withinBand`, `tooIntense`, `tooFlat`) over recent runs.[78] [77]

A runtime policy in CHATDIRECTOR then uses this index at run start: if the last N runs for a given persona/profile combination skew heavily toward `tooIntense`, it automatically down-routes new runs by clamping DET and CDL targets in the requested runContract to gentler presets (e.g., lowering `metricTargets.DET.max` or raising minimum cooldown ratios), while leaving consent tier unchanged. The same mechanism can nudge ARR and UEC / EMD bands based on repeated `tooFlat` or disengaged outcomes, but all adjustments must still remain within consent / profile caps and be logged in telemetry so governance can review adaptation behavior.[76] [77] [78]

94 – Using requiredCooldownRatio for sanctuaries

Director comfort policies already define `requiredCooldownRatio.lowDetMinutes` vs `highDetMinutes`, giving you a numeric floor for sanctuary time per unit of high-DET exposure. At runtime, H.Run (or a small scheduler helper) tracks a rolling window of turn kinds (peak / cooldown / neutral) and computes the current cooldown ratio; when this falls below the director's required ratio, H.Director.constrainMetrics and H.Selector are forced into a "cooldown mode".[77] [76]

In cooldown mode, selectors favor patterns and nodes tagged as safe rooms, hubs, or low-DET moods, and templates shift to lower DET and CDL bands while keeping UEC and ARR within contract bands so sanctuaries remain thematically aligned rather than bland. Directors must also acknowledge sanctuary beats in-world (per your design guidance), using

tone and symbolism that match the region's invariants (CIC, AOS, SHCI) so these segments feel like eerie reprieves within the same horror world, not out-of-genre breaks.[78] [76] [77]

95 – Research-tier-only runtime behaviors

Your tiering and Dead-Ledger design already assumes that some behaviors are restricted to "research" routes and require explicit entitlement. At runtime, the age-tier router and Policy.DeadLedger client can enforce that only research-tier sessions with appropriate proofs can enable certain mapping kernels or BCI-driven modulation modes, by requiring valid `decisionTokenId` for these features before activation.[78]

Concretely, research-only behavior could include:

- Experimental selector kernels (e.g., alternative scoring or routing rules) that dynamically change how nodes are ranked based on live telemetry.
- BCI-coupled modulation beyond the "hint only" policy used elsewhere: for example, stronger DET/CDL downshifts based on distress signals, or experimental SHCI / HVF mappings, still bounded by SAF and consent caps.[76] [78]

These capabilities should be guarded by explicit flags in runContract or experienceContract (e.g., `researchFeatures.enabled`) and hard-gated in H.Run / H.Selector / H.Metrics by tier checks, so they can never be invoked in standard or mature tiers even if code paths exist. Telemetry for research sessions should also be richer (capturing BCI hints, experimental kernel IDs, etc.) to support analysis while public-tier logs remain minimal and privacy-preserving.[77] [78]

⁂

96. `[RUNTIME] (Engine)` How should we structure runtime experiments (A/B tests) around director strategies, selector patterns, and node scoring tweaks so that changes to metric mappings can be evaluated without risking large shifts in user comfort?

97. `[RUNTIME] (Engine)` What runtime safeguards should exist so that if H. modules or external services fail (e.g., selector unavailable), AI-Chat defaults to a clearly safe, low-intensity fallback mode rather than silently continuing with incomplete constraints?
98. `[RUNTIME] (Engine)` How should AI-Chat adapt its pacing (frequency of messages, length of turns) based on current metrics and comfort policy—for example, slowing down during high DET peaks and speeding up in low-DET cooldowns?
99. `[RUNTIME] (Engine)` What runtime logic should be used to mark specific nodes or patterns as "unlocking" new director persona behaviors (e.g., deeper insight into a region's history)

while ensuring those behaviors remain bounded by the persona's invariant and metric envelopes?

100. `[RUNTIME] (Engine)` How should selectors and directors cooperate to ensure that final beats of a run (epilogue, last node) satisfy both metric targets for ARR/CDL and the requirement that all safety modules have fully de-escalated DET to within comfort policies before exit?

Runtime experiments and safeguards should treat director strategies, selector behavior, pacing, unlocks, and endings as parametrized variations on the same H. surfaces, with tight caps from comfort policies and consent so no experiment can move DET/CDL or ARR far outside existing envelopes.[79] [80]

96 – Structuring runtime experiments safely

Runtime A/B tests should be encoded as small, explicit "strategy variants" rather than ad-hoc code paths, and each variant must declare its own metric deltas and caps inside the run and director envelopes. For example, a director strategy variant might say "slightly higher UEC rise per turn, slightly faster cooldown" and be identified in telemetry by a variantId, while H.Director.constrainMetrics continues to clamp UEC/EMD/STCI/CDL/ARR into the same persona + consent bands, preventing large comfort shifts even if the variant tries to push harder.[81] [80] [79]

Selectors and node scoring tweaks can follow the same pattern by treating changes as configuration in history-selector-pattern or scoring rules, gated by experiment IDs that are looked up at runtime. Every turn, H.Selector.selectPattern and H.Node.choosenext still run through the same invariant and metric-aware pipeline, but they read alternative weights or tie-break rules keyed by experiment state; telemetry then logs experimentId alongside selectorDecision, metric snapshots, and comfortOutcome so offline analysis can compare variants without ever having allowed them to exceed safe DET/CDL bands.[81] [79]

97 – Safeguards and safe fallback behavior

If any H. module or external service needed for governed horror fails (selector, director, budget, safety, DeadLedger, metrics), runtime must treat that as a hard safety boundary and drop into a **non-horror fallback** rather than continuing with partial constraints. The unified H. API already standardizes an `ok/data/error` envelope; when calls like H.Selector.selectPattern or H.Director.loadPersona return `ok=false`, AI-Chat should immediately swap to a safe persona or generic narrative mode that ignores dungeon maps, spectral entities, and horror selectors, and only uses low-DET, low-CDL templates that are explicitly tagged as safe.[80] [81]

You can encode this as a simple runtime policy layer: every turn, the engine orchestrator calls H.Consent, H.Budget, H.Selector, H.Safety, and H.Director in a fixed order, and if any required call fails or returns an incompatible state, a "horrorDisabled=true" flag is set on the session. While that flag is true, selectors and node navigation are bypassed, all decisions about intensity and imagery are clamped to minimal DET and CDL, and no DeadLedger-gated

bundles or research-tier kernels can be activated, ensuring the system never improvises horror content outside its full constraint stack.[79] [80]

98 – Adapting pacing from metrics and comfort policy

Pacing should be treated as another surface controlled by metrics and comfort policies rather than a free stylistic choice, with directors reading DET, CDL, and ARR to adjust message frequency and turn length. H.Director.constrainMetrics already computes effective metric bands per session; a small pacing policy can map those bands to timing envelopes, such as "high DET or high CDL ⇒ longer delays between beats, shorter textual turns; low DET cooldown segments ⇒ quicker responses and more expansive descriptions," keeping exposure at peaks from feeling relentless.[80] [81]

At runtime, AI-Chat can compute a per-turn pacing profile from the current metric snapshot (via H.Metrics) plus the director's comfort policy (maxDetPerSession, requiredCooldownRatio), and then have the chat orchestrator and engine plugins enforce this profile by scheduling message delays and gating optional elaboration. This makes pacing adjustments predictable and auditable in telemetry, since each turn's logged metrics and pacing mode can be correlated to comfortOutcome bands, and any future tuning of pacing can still be done under the same DET/CDL caps.[79] [80]

99 – Unlocking new persona behaviors safely

"Unlocks" should be modeled as data-driven state transitions on the director persona rather than ad-hoc branching, with each unlock describing which behaviors are enabled and which additional envelopes they use, all still bounded by the persona's invariant and metric bands. For example, a node or selector pattern might carry an `unlockPersonaFacetId` that, when visited, allows the director to reveal deeper regional history or more complex interpretive tools, but only if the resulting metric targets still fit within the director's invariantEnvelope and metricEnvelope defined at session load.[81] [80]

Runtimewise, H.Director.loadPersona can compute a base persona state plus a set of locked facets; when H.Node.choosenext or H.Selector.selectPattern hits a node or pattern tagged as an unlock, it dispatches a small event to H.Director, which in turn merges the facet's behavior flags and optional target shifts, runs them through H.Director.constrainMetrics, and either accepts or rejects the unlock. This ensures unlocked behaviors never widen DET or CDL bands beyond the persona's comfort policy or consent caps, and that SHCI, CIC, and other invariants still gate how "deep" the director can go into a region's history.[80] [81] [79]

100 – Coordinating endings: ARR/CDL and safe de-escalation

Final beats should be treated as an explicit run stage with its own metric and safety requirements, not just "the last node," and both selectors and directors must target that stage's ARR and CDL bands while ensuring DET is fully de-escalated within comfort policies. A runContract or experience contract can declare an "epilogue stage" with specific target bands such as "ARR above X for ambiguous endings, CDL below Y to avoid lingering

dissonance, DET near zero," and the director persona's comfort policy can add the rule "all exits must satisfy DET ≤ baseline and CDL within allowedCdlSpan." [79] [80]

At runtime, when H.Run determines that only one or two steps remain, H.Selector.selectPattern should filter for patterns tagged as epilogue-capable and whose entertainmentTargets, when combined with current metrics, move ARR and CDL into the desired bands without raising DET. In parallel, H.Director.constrainMetrics and H.Safety.evaluateTurn clamp any remaining intensity, and the session only completes when a turn's metrics snapshot shows DET inside comfortPolicy limits and safety has applied any needed softening or implication, ensuring endings meet both narrative ambiguity targets and hard de-escalation requirements before exit. [81] [80] [79]

⁂

1. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
2. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
3. this-research-focuses-on-under-izA4GL6tQVusFoK8IYnvYA.md
4. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
5. this-research-focuses-on-creat-ljc90qNWSjupFy2TJC2IGQ.md
6. break-down-the-directory-in-up-hsLbuD2NRu29.S35fpB6Yg.md
7. this-research-focuses-on-enhan-2EXdmBbbTneisQNtSJfTWA.md
8. this-research-focuses-on-stren-hZWp2wBpQkW2xDO2IMK_LA.md
9. this-research-focuses-on-creat-b.QYu2gNRjaV4nBZ03r88w.md
10. this-research-focuses-on-compl-E8Y0Lhm_STKi3dQhIJsfSA.md
11. this-research-focuses-on-impro-YmKWzHr3TC6EZLDY.tmO1w.md
12. 1-should-the-research-prioriti-tQnn6sdDQ06XDNmNoVKx.g.md
13. this-research-focuses-on-advan-Gn_aMO_jQvqfg7wXixdLTw.md
14. suggest-the-next-task-below-in-BvOUg9mxQKKnfOeCjfxKsg.md
15. CHAT_DIRECTOR v1 Implementation Research — Module Specifications.docx
16. the-algorithmic-architecture-o-A7N0Cgi2QFqkAHFdhxjM2g.md
17. this-research-focuses-on-desig-DemATE1ZRtOBxLRQlhB93g.md
18. this-research-focuses-on-creat-IQAMBfJDSAit.SxDaXO6Gg.md
19. this-research-focuses-on-impro-RixUoveJRciBLt4RY0p1Vw.md
20. this-research-focuses-on-devel-Eh3ThsQBQg.r6HDAZZhx5w.md
21. formal-definition-and-integrat-V8UK1uYHRKez9vQ0Xfga2A.md
22. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
23. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
24. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
25. this-research-focuses-on-advan-Gn_aMO_jQvqfg7wXixdLTw.md

26. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
27. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
28. this-research-focuses-on-advan-Gn_aMO_jQvqfg7wXixdLTw.md
29. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
30. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
31. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
32. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
33. this-research-focuses-on-advan-Gn_aMO_jQvqfg7wXixdLTw.md
34. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
35. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
36. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
37. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
38. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
39. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
40. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
41. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
42. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
43. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
44. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
45. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
46. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
47. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
48. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
49. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
50. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
51. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
52. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
53. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
54. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
55. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
56. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
57. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
58. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
59. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
60. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
61. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md

62. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
63. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
64. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
65. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
66. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
67. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
68. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
69. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
70. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
71. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
72. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
73. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
74. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
75. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
76. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md
77. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
78. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
79. 1-research-questions-and-items-iqkrJm1mT.OrdJYzXGfBjA.md
80. compiled-report-of-todos-missi-lHKqTuY4Q8O.bRJJ6b1a4w.md
81. which-of-these-memory-architec-dnlg0pjJTqu4DFJ3.oFZgw.md